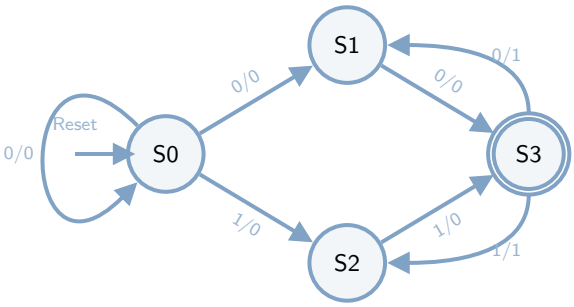
Introduction à l' Electronique

NUMÉRIQUE

Travaux Dirigés – 1ère année

Jean-Christophe Le Lann

Livret solutions



Année académique 2026-2027

Version 2.0

TD 1 – Électronique Numérique

Représentation des nombres et Logique combinatoire

Notes et Conseils

- Le premier objectif de ce TD est de manipuler différentes représentations des *nombres* et notamment de maîtriser les changements de base **sans calculatrice**.
- En dehors du TD, il est suggéré de vérifier que vous êtes suffisamment adroit pour les réaliser également *avec* calculatrice ou à l'aide de votre langage de programmation préféré.
- Le second objectif est de se familiariser avec l'algèbre de Boole.
- *Note : Tous les exercices ne seront pas nécessairement corrigés lors des séances de TD.*

Première partie

Représentation des nombres

1 Compter en base 2 et base 16

1. Compter de 0 à 15 en binaire, puis en hexadécimal. *(On conservera ces valeurs dans un tableau pour la durée de cette partie du TD)*

Solution

Décimal	Binaire	Hexadécimal
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

2. Écrire les puissances de 2^n pour n variant de 0 à 10. En déduire une valeur approchée de $\log_{10} 2$, puis $\log_{10} 5$. Avec un procédé similaire, trouver une valeur approchée de $\log_{10} 3$.

Solution

n	2^n
0	1
1	2
2	4
3	8
4	16
5	32
6	64
7	128
8	256
9	512
10	1024

On remarque que $2^{10} = 1024 \approx 10^3$. D'où :

$$\log_{10}(2^{10}) \approx \log_{10}(10^3) \implies 10 \log_{10} 2 \approx 3 \implies \log_{10} 2 \approx \frac{3}{10} = 0.3$$

(En réalité, $\log_{10} 2 \approx 0.301$ à 10^{-3} près. L'erreur est de 0.3%).
De même :

$$5 = \frac{10}{2} \implies \log_{10} 5 = \log_{10} 10 - \log_{10} 2 \approx 1 - 0.3 = 0.7$$

Pour $\log_{10} 3$, on peut remarquer que $3^4 = 81 \approx 80 = 10 \times 2^3 \times 1$ (ou $3^2 \approx 10$). Une approximation courante utilise $3^4 \approx 10 \times 8 = 10 \times 2^3$:

$$4 \log_{10} 3 \approx 1 + 3 \log_{10} 2 \approx 1 + 3(0.3) = 1.9 \implies \log_{10} 3 \approx \frac{1.9}{4} = 0.475$$

(Valeur réelle ≈ 0.477).

3. Calculer la valeur décimale de 10101011_2 en passant par les puissances de 2, puis en passant par les puissances de 16.

Solution

$$\begin{aligned} 10101011_2 &= 2^7 + 2^5 + 2^3 + 2^1 + 2^0 \\ &= 128 + 32 + 8 + 2 + 1 = \mathbf{171_{10}} \end{aligned}$$

En passant par l'hexadécimal (groupement par 4 bits : 1010 et 1011) :

$$\begin{aligned} 1010\ 1011_2 &= \mathrm{AB}_{16} \\ &= 10 \times 16^1 + 11 \times 16^0 \\ &= 160 + 11 = \mathbf{171_{10}} \end{aligned}$$

4. Écrire les puissances de 2^{-n} pour n variant de 1 à 4.

Solution

n	2^{-n}
1	0,5
2	0,25
3	0,125
4	0,0625

Si on s'attribue 2 bits pour représenter (quantifier) une partie décimale, on a les équivalences suivantes :

Binaire	Décimal
00	0,00
01	0,25
10	0,50
11	0,75

Pour 3 bits, on double la précision (pas de 0,125).

Remarque : Des nombres possédant une partie décimale sont transformés ici en de simples *entiers*, en supposant que la position de la virgule est fixée à l'avance. Cette représentation est appelée **virgule fixe** (*fixed-point*). Elle est très rapide à traiter par l'électronique, à coût matériel très faible, et est très utilisée en traitement du signal embarqué (DSP). À l'inverse, une représentation en **virgule flottante** est plus consommatrice de ressources (surface) et de temps, mais offre une plage dynamique et une précision beaucoup plus importantes (norme IEEE 754 traitée dans l'exercice 4).

2 Conversions entre bases

1. Convertir le nombre 39_{10} en binaire.

Solution

On effectue les divisions successives par 2 :

$$\begin{aligned} 39 &= 2 \times 19 + 1 \\ 19 &= 2 \times 9 + 1 \\ 9 &= 2 \times 4 + 1 \\ 4 &= 2 \times 2 + 0 \\ 2 &= 2 \times 1 + 0 \\ 1 &= 2 \times 0 + 1 \end{aligned}$$

En lisant les restes de bas en haut : $39_{10} = \mathbf{100111_2}$.

2. Convertir le nombre $(0,375)_{10}$ en binaire.

Solution

On effectue les multiplications successives par 2 :

$$\begin{aligned} 0,375 \times 2 &= 0,75 \quad \text{(partie entière 0)} \\ 0,75 \times 2 &= 1,50 \quad \text{(partie entière 1)} \\ 0,50 \times 2 &= 1,00 \quad \text{(partie entière 1)} \end{aligned}$$

En lisant les parties entières de haut en bas : $0,375_{10} = \mathbf{0,011_2}$. (On remarque que $0,375 = 3 \times 0,125 = 3 \times 2^{-3}$).

3. En déduire la représentation binaire du nombre $(39,125)_{10}$.

Solution

On combine les résultats précédents (en notant que $0,125 = 0,001_2$) :

$$(39,125)_{10} = \mathbf{100111,001_2}$$

4. Convertir le nombre $(0,2)_{10}$ en binaire. Que remarquez-vous ?

Solution

On effectue les produits successifs :

$$\begin{aligned} 0,2 \times 2 &= 0,4 \quad (0) \\ 0,4 \times 2 &= 0,8 \quad (0) \\ 0,8 \times 2 &= 1,6 \quad (1) \\ 0,6 \times 2 &= 1,2 \quad (1) \\ 0,2 \times 2 &= 0,4 \quad (0) \quad \leftarrow \text{On retombe sur le début !} \end{aligned}$$

On trouve $0,2_{10} = \mathbf{0,00110011\ldots_2} = 0,\overline{0011}_2$. *Remarque :* Une fraction décimale finie peut avoir une représentation binaire infinie périodique, ce qui explique les erreurs d'arrondi en informatique.

5. Convertir $(345,158)_{10}$ en octal.

Solution

Partie entière (divisions successives par 8) :

$$\begin{aligned} 345/8 &= 43 \quad \text{reste } \mathbf{1} \\ 43/8 &= 5 \quad \text{reste } \mathbf{3} \\ 5/8 &= 0 \quad \text{reste } \mathbf{5} \end{aligned}$$

$\Longrightarrow 345_{10} = 531_8$

Partie décimale (multiplications successives par 8) :

$$\begin{aligned} 0,158 \times 8 &= 1,264 \quad (\mathbf{1}) \\ 0,264 \times 8 &= 2,112 \quad (\mathbf{2}) \\ 0,112 \times 8 &= 0,896 \quad (\mathbf{0}) \\ 0,896 \times 8 &= 7,168 \quad (\mathbf{7}) \end{aligned}$$

$\Longrightarrow 0,158_{10} \approx 0,1207_8$.
Résultat final : $\mathbf{531,1207_8}$.

6. Convertir $(389)_{10}$ en hexadécimal.

Solution

Divisions successives par 16 :

$$\begin{aligned} 389/16 &= 24 \quad \text{reste } \mathbf{5} \\ 24/16 &= 1 \quad \text{reste } \mathbf{8} \\ 1/16 &= 0 \quad \text{reste } \mathbf{1} \end{aligned}$$

Résultat : $\mathbf{185_{16}}$ (ou `0x185`).

7. Convertir 011001111011101_2 en hexadécimal.

Solution

On groupe les bits par paquets de 4 en partant de la droite (on peut ajouter un 0 à gauche) :

$$0110\ 0111\ 1011\ 1101_2$$

$$\Downarrow$$

$$6 \quad 7 \quad \mathrm{B} \quad \mathrm{D}_{16}$$

Résultat : $\mathbf{67BD_{16}}$. *(Note : le corrigé original indiquait 33DD, ce qui correspondait à un regroupement erroné. Le regroupement correct donne 67BD).*

8. Convertir $(10110001101011,1111)_2$ en octal.

Solution

On groupe les bits par paquets de 3 en partant de la virgule :

```
010 110 001 101 011 . 111 100
  2   6   1   5   3 .   7   4
```

Résultat : $\mathbf{26153,74_8}$.

9. Convertir $(10110001101011, 111101)_2$ en hexadécimal.

Solution

On groupe les bits par paquets de 4 :

```
0010 1100 0110 1011 . 1111 0100
   2    C    6    B .    F    4
```

Résultat : $\mathbf{2C6B,F4_{16}}$.

10. Le nombre 101110001_2 est écrit en BCD (décimal codé binaire). Convertir le nombre décimal obtenu en binaire pur. Combien de bits sont alors nécessaires ? Conclure.

Solution

En BCD, on groupe par 4 bits pour lire les chiffres décimaux :

```
0001  0111  0001
  1     7     1  => 171 en décimal
```

Conversion de 171_{10} en binaire pur : $171 = 128+32+8+2+1 = 2^7+2^5+2^3+2^1+2^0$ $\Longrightarrow$ $\mathbf{10101011_2}$.
Il faut **8 bits** en binaire pur, contre **12 bits** (3×4) en BCD.
Conclusion : Le BCD est moins compact et moins efficace pour le calcul, mais il est très utile pour l'affichage direct sur des afficheurs 7 segments, car chaque digit est isolé.

3 Nombres signés en complément à 2

Rappels

Jusqu'ici, nous nous sommes intéressés aux nombres positifs. Il existe plusieurs manières de représenter un nombre négatif n. La manière la plus intuitive serait de dédier un bit (le plus à gauche) pour préciser le signe (ex : 0 pour $+$, 1 pour $-$). Malheureusement, cette représentation possède deux défauts majeurs : il existe deux représentations du zéro ($+0$ et -0) et il faut modifier l'algorithme d'addition.

Les architectes des ordinateurs ont retenu une autre représentation plus pratique : le **complément à 2**. C'est cette représentation qui régit le fonctionnement des *unités arithmétiques et logiques* (ALU) des ordinateurs.

Avec k bits, on peut représenter l'ensemble des nombres entiers dans l'intervalle :

$$[-2^{k-1},\ \ldots,\ 2^{k-1}-1]$$

Le complément à 2 d'un entier n est obtenu par le calcul de la quantité $2^k - n$. On peut vérifier aisément que :

— Le complément à 2 du complément à 2 d'un entier n est n lui-même.
— Le complément à 2 de zéro est zéro (on néglige la retenue qui dépasse la taille k fixée).

On doit retenir la formule de passage en complément à 2 d'un nombre négatif $-X$:

$$-X = \overline{X} + 1 \tag{1}$$

La barre figurant au-dessus du X signifie "inversion de tous les bits" (complément à 1). Si l'entier n est codé sur k bits $(s_{k-1} \dots s_0)$ en complément à 2, sa valeur décimale est :

$$n_{10} = -s_{k-1}2^{k-1} + \sum_{i=0}^{k-2} s_i 2^i \tag{2}$$

Exercices

1. Déterminer la plage des valeurs des entiers signés sur $k = 4$ bits. Représenter ces valeurs en binaire. Quelles sont les représentations des valeurs extrêmes ? de 0, 1, et -1 ?

Solution

Avec $k = 4$ bits, la plage est $[-2^{4-1},\ 2^{4-1} - 1] = [-\mathbf{8},\ \mathbf{7}]$.

Décimal	Binaire (Cà2)	Décimal	Binaire (Cà2)
-8	**1000**	0	0000
-7	1001	1	0001
-6	1010	2	0010
-5	1011	3	0011
-4	1100	4	0100
-3	1101	5	0101
-2	1110	6	0110
-1	1111	7	0111

Points clés :
— Les nombres positifs sont codés "normalement".
— Alors qu'en non-signé on pouvait compter jusqu'à +15 avec 4 bits, ici on est limité à +7.
— Les nombres qui présentent un '1' en MSB (Most Significant Bit) sont forcément négatifs dans cette représentation.
— Le codage de la valeur -1 est 1111_2.

2. Quel est le complément à 2 du nombre 8, codé sur 6 bits ? Utiliser la formule (1) et vérifier le résultat en appliquant la formule (2).

Solution

Formule (1) : On inverse tous les bits et on ajoute 1. $8_{10} = 001000_2$ (sur 6 bits)
Inversion : 110111_2
Ajout de 1 : $110111_2 + 1_2 = \mathbf{111000_2}$.
Formule (2) : Vérification de la valeur de 111000_2 :

$$-1 \times 2^5 + 1 \times 2^4 + 1 \times 2^3 + 0 + 0 + 0 = -32 + 16 + 8 = -\mathbf{8}$$

Le résultat est cohérent.

3. Soustraire : $91_{10} - 108_{10}$ en passant par le complément à 2.

Solution

Attention ! 91 et 108 peuvent se représenter sur 7 bits, mais pas -108. Il faut donc passer à **8 bits** pour éviter les débordements.

$$\begin{aligned} 91_{10} &= 64+16+8+2+1 = 01011011_2 \\ 108_{10} &= 64+32+8+4 = 01101100_2 \\ -108_{10} &= \overline{01101100}+1 = 10010011+1 = 10010100_2 \end{aligned}$$

Addition en complément à 2 :

```
  01011011  ( 91)
+ 10010100  (-108)
------------
  11101111
```

Vérification du résultat 11101111_2 :

$$-2^7+(2^6+2^5+2^3+2^2+2^1+2^0) = -128+(64+32+8+4+2+1) = -128+111 = \mathbf{-17}$$

Le calcul est correct ($91-108=-17$).

4. Écrire les nombres suivants en binaire sur 6 bits : $+24, +31, -24, -31$. Passez ensuite à 7 bits. Que remarquez-vous ?

Solution

Sur 6 bits :
- $+24 = 011000$
- $+31 = 011111$
- $-24 = \overline{011000}+1 = 100111+1 = 101000$
- $-31 = \overline{011111}+1 = 100000+1 = 100001$

Sur 7 bits :
- $+24 = 0011000$
- $+31 = 0011111$
- $-24 = \overline{0011000}+1 = 1100111+1 = 1101000$
- $-31 = \overline{0011111}+1 = 1100000+1 = 1100001$

Remarque : On observe que le bit de signe se **propage** vers la gauche (c'est l'extension de signe). Pour passer d'une taille à une autre, il suffit de dupliquer le MSB.

5. Soustraire : $1100\ 0101\ 0100\ 0000_2 - 101000\ 1111\ 1111_2 - 110\ 1111\ 1110_2 - 10\ 1110_2$.

Solution

Piège classique : Si on ne tient pas compte du format et qu'on complémente "mal" (sans aligner les tailles), on obtient un résultat faux. Il faut ramener tous les opérandes à la même taille (ici, 16 bits minimum, mais 17 bits pour être sûr avec des signés).
Valeurs décimales (pour vérification) : $50496 - 10495 - 1790 - 46 = 38165$.
En se ramenant à une représentation **signée sur 17 bits** (pour gérer correctement le bit de signe de l'opérande positif initial de 16 bits) :

```
  0 1100 0101 0100 0000  (+50496)
+ 1 1101 0111 0000 0001  (-10495 en C2 sur 17 bits)
+ 1 1111 1001 0000 0010  (-1790 en C2 sur 17 bits)
+ 1 1111 1111 1101 0010  (-46 en C2 sur 17 bits)
-----------------------
1 0 1001 0101 0001 0101
```

La retenue de 18ème bit est ignorée. Le résultat sur 17 bits est $0\ 1001\ 0101\ 0001\ 0101_2$, qui vaut bien $\mathbf{38165_{10}}$.
Règle générale : Pour additionner/soustraire des nombres signés, il faut les coder sur n bits, où :

$$n = 1 + \max\left(\lfloor \log_2(|x_1|) \rfloor + 1,\ \ldots,\ \lfloor \log_2(|x_m|) \rfloor + 1\right)$$

Ici, $n = 1 + \max(16, 14, 11, 6) = 17$ bits.

4 Norme IEEE 754

Rappel : norme IEEE 754 (simple précision, 32 bits) Un flottant est codé par trois champs : 1 bit de signe s, 8 bits d'exposant e et 23 bits de mantisse m. La valeur représentée est, pour la plupart des nombres :

$$V = (-1)^s \times 2^{e-127} \times (1 + m)$$

où $m = \sum_{i=1}^{23} b_i 2^{-i}$ est la partie fractionnaire. L'exposant est représenté en excédent (biais) à 127. La mantisse est toujours normalisée avec un 1 implicite (gain de place), sauf pour les cas particuliers suivants :

- si $e = 255$ et $m = 0$, alors $V = \pm\infty$ (selon s) ;
- si $e = 255$ et $m \neq 0$, alors $V =$ `NaN` (Not a Number) ;
- si $e = 0$ et $m = 0$, alors $V = \pm 0$;
- si $e = 0$ et $m \neq 0$, il s'agit d'un nombre dénormalisé, dont la valeur est $V = (-1)^s \times 2^{-126} \times m$.

Exercice : Représenter le nombre $238,4375$ dans sa représentation IEEE 754 (32 bits) en virgule flottante.

Solution

1. Conversion en binaire :
Partie entière : $238_{10} = 128 + 64 + 32 + 8 + 4 + 2 = \mathbf{11101110_2}$
Partie décimale : $0,4375 = 0,25 + 0,125 + 0,0625 = 2^{-2} + 2^{-3} + 2^{-4} = \mathbf{0,0111_2}$
Donc : $238,4375_{10} = \mathbf{11101110,0111_2}$.
2. Normalisation :
On déplace la virgule pour obtenir un seul '1' avant la virgule :

$$11101110,0111_2 = 1,11011100111_2 \times 2^7$$

L'exposant réel est $n = 7$.
3. Codage des champs :
- **Signe (s) :** Le nombre est positif, donc $s = \mathbf{0}$.
- **Exposant (e) :** $e = n + 127 = 7 + 127 = 134_{10}$.
  $134_{10} = 128 + 4 + 2 = \mathbf{10000110_2}$.
- **Mantisse (m) :** On prend la partie fractionnaire après la virgule (11011100111) et on complète avec des zéros à droite pour atteindre 23 bits :
  110 1110 0111 0000 0000 000$_2$.

4. Assemblage final (32 bits) :

```
 S |    Exposant (8)   |          Mantisse (23)          |
 0 | 1 0 0 0 0 1 1 0 | 1 1 0 1 1 1 0 0 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 |
```

Soit en hexadécimal : **0x436E7000**.

Deuxième partie

Algèbre de Boole et Logique combinatoire

Nous commençons par quelques manipulation élementaires autour de la minimisation d'expressions booléennes et leur mise en forme canonique, à l'aide de l'algèbre de Boole. Nous illustrons également la méthode des tableaux de Karnaugh qu'il faut voir comme une technique tabulée de minimisation astucieuse qu'il est intéressant de connaître pour de petits circuits. Nous poursuivons par la construction d'opérateurs matériels élémentaires : additionneurs, multiplexeurs, comparateurs.

5 Quelques formulations booléennes

Écrire sous la première forme canonique (somme de mintermes) les fonctions définies par les propositions suivantes :

1. $F(a,b,c) = 1$ si et seulement si aucune des variables a, b, c ne prend la valeur 1.
2. $F(a,b,c) = 1$ si et seulement si au plus une des variables a, b, c prend la valeur 0.

Solution

1. $f(A,B,C) = \overline{A}\,\overline{B}\,\overline{C}$

2. $f(A,B,C) = \overline{A}BC + A\overline{B}C + AB\overline{C} + ABC$

6 Minimisation d'expression booléennes

Simplifier algébriquement les fonctions suivantes :

1. $F = \overline{x}\overline{y} + xy + \overline{x}y$
2. $F = (x + \overline{y})(x\overline{y} + z)z$

Solution

1. $F = \overline{x}(\overline{y} + y) + xy = \overline{x}(1) + xy = \overline{x} + xy = (\overline{x} + x)(\overline{x} + y) = \mathbf{\overline{x} + y}$
2. $F = (x + \overline{y})(x\overline{y}z + z) = (x + \overline{y})z(x\overline{y} + 1) = (x + \overline{y})z(1) = \mathbf{(x + \overline{y})z}$

7 Simplifications par tableaux de Karnaugh

Simplifier, par la méthode des tableaux de Karnaugh, les fonctions booléennes suivantes. Dessiner le circuit correspondant et déterminer son chemin critique, en supposant que toutes les portes présentent un temps de propagation de 5 ns.

7.1 K-map à 3 variables

1. $F(a, b, c) = \overline{a}\overline{b}c + \overline{a}b\overline{c} + ab\overline{c}$
2. $F(a, b, c) = \overline{a}b\overline{c} + \overline{a}bc + ab\overline{c}$
3. $F(a, b, c) = \overline{a}\overline{b}c + a\overline{b}\overline{c} + ab\overline{c}$, sachant que la valeur de F pour les combinaisons $\overline{a}bc$ et abc est indifférente (donc care X).

Solution

1. $F(a, b, c) = \overline{a}\overline{b}c + b\overline{c}$
2. $F(a, b, c) = \overline{a}b + b\overline{c} = b(\overline{a} + \overline{c})$
3. $F(a, b, c) = \overline{a}c + a\overline{c} = a \oplus c$ (en utilisant les cares pour former des groupes de 2 optimaux).

7.2 K-map à 4 variables

1. $F(a, b, c, d) = \overline{a}b\overline{c}\overline{d} + \overline{a}b\overline{c}d + \overline{a}bcd + \overline{a}bc\overline{d}$
2. $F(a, b, c, d) = \overline{a}\overline{b}\overline{c}d + \overline{a}\overline{b}cd + a\overline{b}\overline{c}d + a\overline{b}cd$

Solution

1. En factorisant ou via le tableau, on isole $a = 0$ et $b = 1$: $\mathbf{F = \overline{a}b}$
2. On isole $b = 0$ et $d = 1$: $\mathbf{F = \overline{b}d}$

8 Équations logiques de circuits de base

8.1 Additionneur

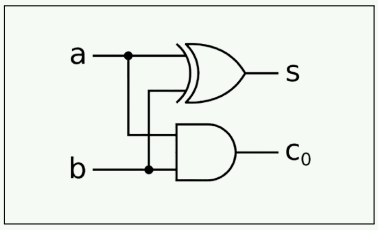
1. Établir la table de vérité d'un *demi-additionneur* logique, puis les deux équations logiques associées : ce circuit additionne simplement 2 entrées booléennes a et b (donc *sans retenue entrante*). Il calcule deux sorties : la somme s et la retenue sortante c_o. Dessiner le circuit.

Solution

a	b	s	c_o
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

On trouve les équations suivantes :

$$s = a \oplus b$$
$$c_o = a \cdot b$$



2. Établir la table de vérité d'un *additionneur complet* pour deux opérandes 1 bit. Ce dernier tient désormais compte d'une retenue entrante c_i. Établir les équations simplifiées des sorties, puis dessiner le circuit correspondant. Au préalable, on démontrera que : $\overline{x \oplus y} = x \oplus \overline{y}$.

Solution

Table de vérité :

a	b	c_i	s	c_o
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Formes brutes :

$$s = \overline{a}\overline{b}c_i + \overline{a}b\overline{c_i} + a\overline{b}\overline{c_i} + abc_i$$
$$c_o = \overline{a}bc_i + a\overline{b}c_i + ab + \overline{c_i} + abc_i \quad \text{(Note : forme brute à simplifier)}$$

Simplification :

$$s = \overline{a}(b \oplus c_i) + a(b \oplus \overline{c_i})$$
$$c_o = a \cdot b + c_i(a \oplus b)$$

Démontrons que $\overline{x \oplus y} = x \oplus \overline{y}$:

$$\begin{aligned}\overline{x \oplus y} &= \overline{x\overline{y} + \overline{x}y} \\ &= (\overline{x} + y)(x + \overline{y}) \quad \text{(De Morgan)} \\ &= \overline{x}x + \overline{x}\overline{y} + xy + y\overline{y} \\ &= 0 + \overline{x}\overline{y} + xy + 0 \\ &= x \oplus \overline{y}\end{aligned}$$

En posant $\alpha = b \oplus c_i$, on a pour la somme s :

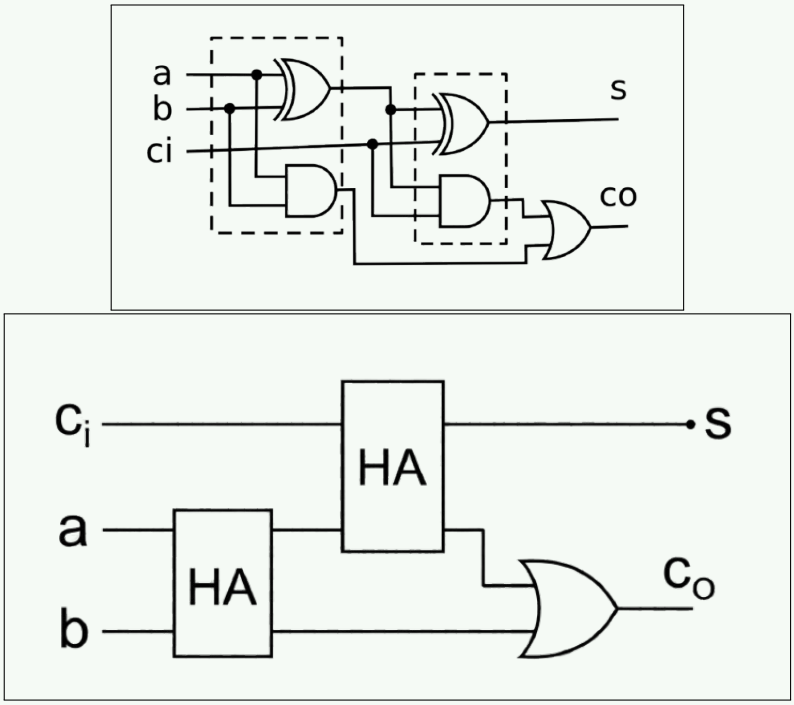
$$s = \overline{a}\alpha + a\overline{\alpha} = a \oplus \alpha = a \oplus b \oplus c_i$$

En résumé :

$$s = a \oplus b \oplus c_i$$
$$c_o = a \cdot b + c_i(a \oplus b)$$

3. Montrer qu'un additionneur complet "1 bit" peut réutiliser le demi-additionneur de la question 1.

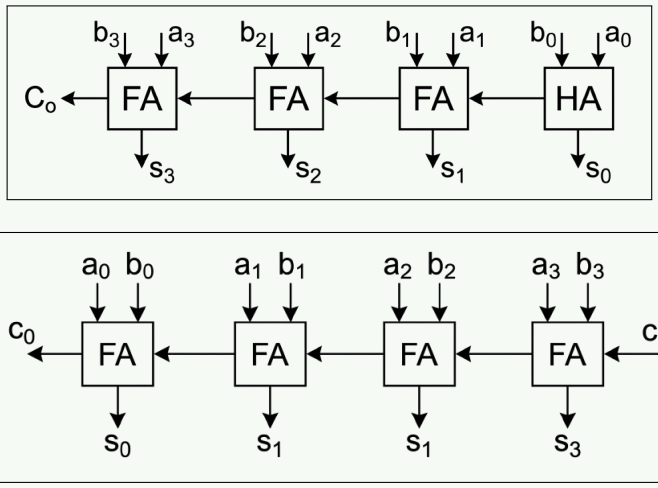
Solution



4. Dessiner un additionneur 4 bits en réutilisant l'additionneur 1 bit.

Solution

On propose ci-dessous 2 solutions : la première ne permet pas l'utilisation de retenue entrante globale, tandis que la seconde utilise une telle retenue. La seconde solution est parfaitement régulière en terme d'aboutement, constituée de full-adders 1 bit en cascade.



5. Quel est le chemin critique de ce dernier circuit ? On rappelle que le chemin critique est le parcours le plus long entre une entrée et une sortie combinatoire. On le mesurera en nombre de portes logiques élémentaires traversées.

Solution

Dans la réalité, chaque porte logique est pré-caractérisée par des temps de traversée donnés par le constructeur (Intel, TSMC, etc.) et il faut sommer des *temps*. L'exercice simplifie cela en comptant le nombre de portes "abstraites".
Nommons les étages de 0 à 3. Deux cas distincts :

(a) **Sans retenue entrante globale :** L'étage 0 a un chemin critique de 3 portes (2 demi-additionneurs + 1 OU). Les étages 1 à 3 ont un chemin critique de c_{in} à c_{out}, soit 2 portes chacun. Total pour les 3 étages : $3 \times 2 = 6$. Chemin critique total : $\mathbf{3 + 6 = 9}$ portes.

(b) **Avec retenue entrante globale :** Le même raisonnement conduit à un chemin critique de **7 portes** traversées (le premier étage étant un full-adder complet dès le départ).

Remarque : structurellement, 9 portes existent dans le premier cas, créant un "faux chemin critique" si l'entrée est forcée à 0.

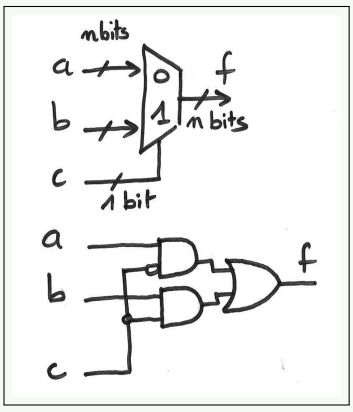
6. Peut-on imaginer d'anticiper le calcul de la retenue ? Si oui, comment ?

8.2 Multiplexeur

La fonction d'un multiplexeur est de sélectionner, grâce à un signal de contrôle, un signal d'entrée parmi plusieurs et de transmettre sa valeur sur une sortie unique.

1. Dessiner le symbole communément admis pour représenter un multiplexeur à 2 entrées binaires (chacune codée sur 1 bit). On parle de multiplexeur "2 vers 1".
2. Calculer l'équation d'un multiplexeur "2 vers 1" pour des entrées codées sur 1 bit.
3. Lorsque les données d'entrées sont codées sur $n > 1$ bits, chacun des bits du signal multiplexé suit évidemment un chemin similaire. Dessiner un tel multiplexeur à 2 entrées, pour $n = 2$.
4. Etablir la structure du multiplexeur à m entrées, à l'aide de multiplexeurs.
5. Calculer le nombre de couches logiques traversées pour un multiplexeur m, n ("m vers 1" et dont les données sont codées sur n bits).

Solution

On parle de multiplexeur 2-vers-1 (2-to-1 MUX). 

Pour n bits, il suffit de mettre n multiplexeurs 2-vers-1 en parallèle, tous pilotés par le même signal de sélection.

8.3 Comparateur

Soit deux données a et b quelconques, encodées par deux variables booléennes générales $(a_{n-1}, \ldots, a_0) \in \mathcal{B}^n$ et $(b_{n-1}, \ldots, b_0) \in \mathcal{B}^n$.

1. Démontrer que le résultat booléen f de comparaison d'égalité entre les deux données vaut :
$$f(a,b) = \prod_{i=0}^{i=n} \overline{a_i \oplus b_i}$$
2. Dessiner la structure du comparateur matériel pour $n = 4$.

9 Formes canoniques

Prérequis : Codage binaire et indexation des combinaisons booléennes
Pour un système à n variables logiques ordonnées, il existe 2^n combinaisons possibles de valeurs. Chaque combinaison peut être interprétée comme un nombre binaire, ce qui permet de lui associer de manière unique un indice entier compris entre 0 et $2^n - 1$. Par exemple, pour trois variables ordonnées (x, y, z), la combinaison de vérités $(1, 0, 1)$ est associée au mot binaire 101_2, qui correspond à l'indice décimal 5 : ce dernier pourra être évoqué comme le monôme m_5. Ce formalisme d'indexation numérique standardise la désignation de n'importe quel état du système et s'applique de manière générale en logique combinatoire, notamment pour l'organisation des tables de vérité, le référencement des cases d'un tableau de Karnaugh, etc.

Formes canoniques et passage de l'une à l'autre
La **première forme canonique** (ou Forme Normale Disjonctive, FND) est une somme de produits appelés mintermes (m_i), où chaque m_i est le produit logique associé à la combinaison d'indice i pour laquelle la fonction vaut 1. À l'inverse, la **deuxième forme canonique** (ou Forme Normale Conjonctive, FNC) est un produit de sommes appelées maxterms (M_i), où chaque M_i est la somme logique associée à la combinaison d'indice i pour laquelle la fonction vaut 0. Le passage d'une forme à l'autre exploite directement la complémentarité des indices au sein de l'ensemble des 2^n combinaisons. La fonction inverse $\bar{f}$ étant constituée des mintermes dont les indices sont absents de f, l'application de la double négation ($f = \overline{\bar{f}}$) et des lois de De Morgan permet de transformer la somme de ces mintermes manquants en un produit de maxterms de mêmes indices :

$$\begin{aligned} f(x,y,z) &= m_1 + m_3 + m_5 \quad \text{(la fonction vaut 1 pour les indices 1, 3 et 5)} \\ \bar{f}(x,y,z) &= m_0 + m_2 + m_4 + m_6 + m_7 \quad \text{(indices restants parmi les } 2^3 \text{ possibles)} \\ f(x,y,z) = \overline{\bar{f}(x,y,z)} &= M_0 \cdot M_2 \cdot M_4 \cdot M_6 \cdot M_7 \quad \text{(par passage au dual via De Morgan)} \end{aligned}$$

Ainsi, commuter entre les deux formes canoniques revient simplement à inverser la nature de l'opérateur principal (somme ou produit) en basculant sur l'ensemble complémentaire des indices de l'univers des combinaisons. *Note : Ces formes ne sont pas simplifiées.*

Établir les deux formes canoniques des fonctions suivantes :

1. $F_1 = xy + yz + xz$
2. $F_2 = x + yz + \overline{yz}t$

Solution

1. $F_1 = XY + YZ + XZ$

X	Y	Z	F_1
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

- Première forme canonique

$$F_1 = \overline{X}YZ + X\overline{Y}Z + XY\overline{Z} + XYZ$$

- Seconde forme canonique

$$F_1 = (X + Y + Z)(X + Y + \overline{Z})(X + \overline{Y} + Z)(\overline{X} + Y + Z)$$

2. $F_2 = X + YZ + \overline{Y}\,\overline{Z}\,T$

X	Y	Z	T	F_2
0	0	0	0	0
0	0	0	1	1
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	1
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	1
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

- Première forme canonique

$$F_2 = \overline{X}\,\overline{Y}\,\overline{Z}T + \overline{X}\,YZ\overline{T} + \overline{X}\,YZT + X\,\overline{Y}\,\overline{Z}\,\overline{T} + X\,\overline{Y}\,\overline{Z}T + X\,\overline{Y}\,Z\overline{T} + X\,\overline{Y}\,ZT + XY\overline{Z}\,\overline{T} + XY\overline{Z}T + XYZ\overline{T} + XYZT$$

- Seconde forme canonique

$$F_2 = (X + Y + Z + T)(X + Y + \overline{Z} + T)(X + Y + \overline{Z} + \overline{T})(X + \overline{Y} + Z + T)(X + \overline{Y} + Z + \overline{T})$$

10 Vu-mètre

On se propose de réaliser un “contrôleur de vu-mètre” : c’est un dispositif qui permet de visualiser une grandeur numérique (un volume sonore par exemple) au moyen d’une barrette lumineuse qui s’étire plus ou moins en fonction de la grandeur.

1. Schématiser le principe d’ensemble dans le cas où la grandeur à visualiser est codée sur 3 bits.
2. À partir du tableau de Karnaugh, trouver une réalisation possible.
3. On suppose que l’on dispose désormais d’un composant complexe de décodage (un décodeur 3 vers 8). Rappeler la table de vérité de ce circuit. Utiliser alors ce décodeur pour proposer une nouvelle réalisation.

Solution

Attention : lorsqu'on envoie le code `000`, on ne veut *aucune* LED allumée. Quand on envoie le code `111` (7), on veut voir 7 LEDs allumées, numérotées de L_0 à L_6 (et non 8).
Table de vérité du Vu-mètre :

e_2	e_1	e_0	L_0	L_1	L_2	L_3	L_4	L_5	L_6
0	0	0	0	0	0	0	0	0	0
0	0	1	1	0	0	0	0	0	0
0	1	0	1	1	0	0	0	0	0
0	1	1	1	1	1	0	0	0	0
1	0	0	1	1	1	1	0	0	0
1	0	1	1	1	1	1	1	0	0
1	1	0	1	1	1	1	1	1	0
1	1	1	1	1	1	1	1	1	1

Simplification (K-map) :

$$
\begin{aligned}
L_0 &= e_2 + e_1 + e_0 \\
L_1 &= e_2 + e_1 \\
L_2 &= \overline{e_2} e_1 + e_2 \overline{e_1} e_0 \quad \text{(ou forme équivalente)} \\
L_3 &= e_2 \\
L_4 &= e_2 e_1 + e_2 \overline{e_1} e_0 \\
L_5 &= e_2 e_1 \\
L_6 &= e_2 e_1 e_0
\end{aligned}
$$

Réalisation avec un décodeur 3 vers 8 :
La sortie C_i d'un décodeur s'allume (vaut 1) uniquement lorsque la valeur binaire d'entrée correspond à i.

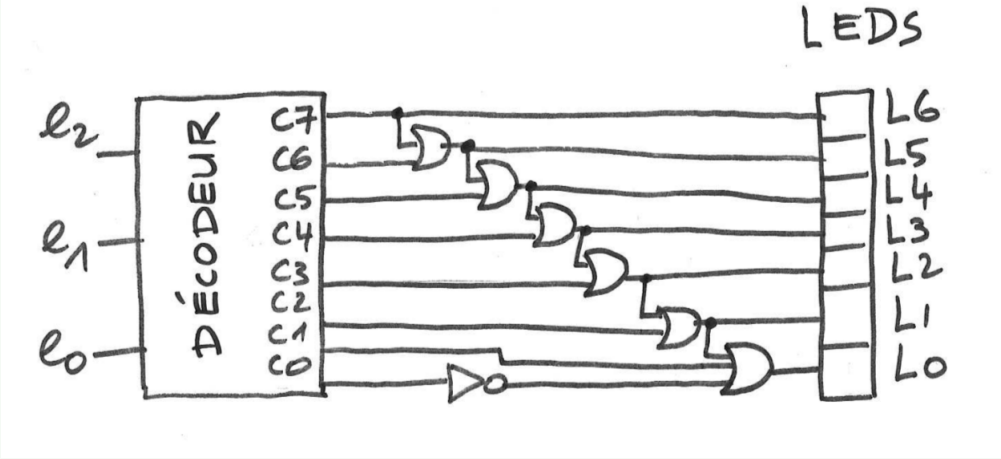
e_2	e_1	e_0	C_0	C_1	C_2	C_3	C_4	C_5	C_6	C_7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

Solution

On remarque que L_k doit être allumée pour toutes les valeurs d'entrée strictement supérieures à k. Ainsi :

$$\begin{aligned}
L_0 &= C_1 + C_2 + C_3 + C_4 + C_5 + C_6 + C_7 \\
L_1 &= C_2 + C_3 + C_4 + C_5 + C_6 + C_7 \\
L_2 &= C_3 + C_4 + C_5 + C_6 + C_7 \\
&\vdots \\
L_5 &= C_6 + C_7 \\
L_6 &= C_7
\end{aligned}$$

On peut réaliser cela en cascade avec des portes OU, ce qui est très élégant et modulaire. Seul le cas du zéro (L_0 éteinte quand $C_0 = 1$) est géré naturellement car C_0 n'est pas inclus dans la somme de L_0.



11 Le jeu des cambrioleurs

Le principe du jeu [1] est le suivant : un jeton est introduit par le joueur en haut du dispositif. Le but est de faire circuler ce jeton dans le dispositif, jusqu'à ce qu'il apparaisse en sortie. Dans l'intervalle, le jeton peut rester coincé à différents endroits (ou "niveaux") : seul un code secret correctement introduit à cet endroit permet au jeton de continuer son parcours. Sinon, l'utilisateur voit le code d'erreur `0xDEAD` apparaître en sortie. Chaque code secret, associé au niveau i, est codé sur 4 bits.

1. Imaginer un circuit combinatoire qui simule ce jeu. On utilisera pour cela différents circuits connus : multiplexeurs et comparateurs. Enrichir votre circuit avec d'autres circuits combinatoires afin d'indiquer le nombre de niveaux franchis correctement.
2. Sachant que l'on peut envoyer 1 million de codes par seconde, combien de temps faudrait-il, au pire cas, pour tester toutes les combinaisons possibles, dans le cas où le nombre de niveaux est 9 ? Refaites le calcul pour des codes sur 5 bits.

Cet exercice ne nécessite pas de formules complexes, mais une réflexion architecturale.

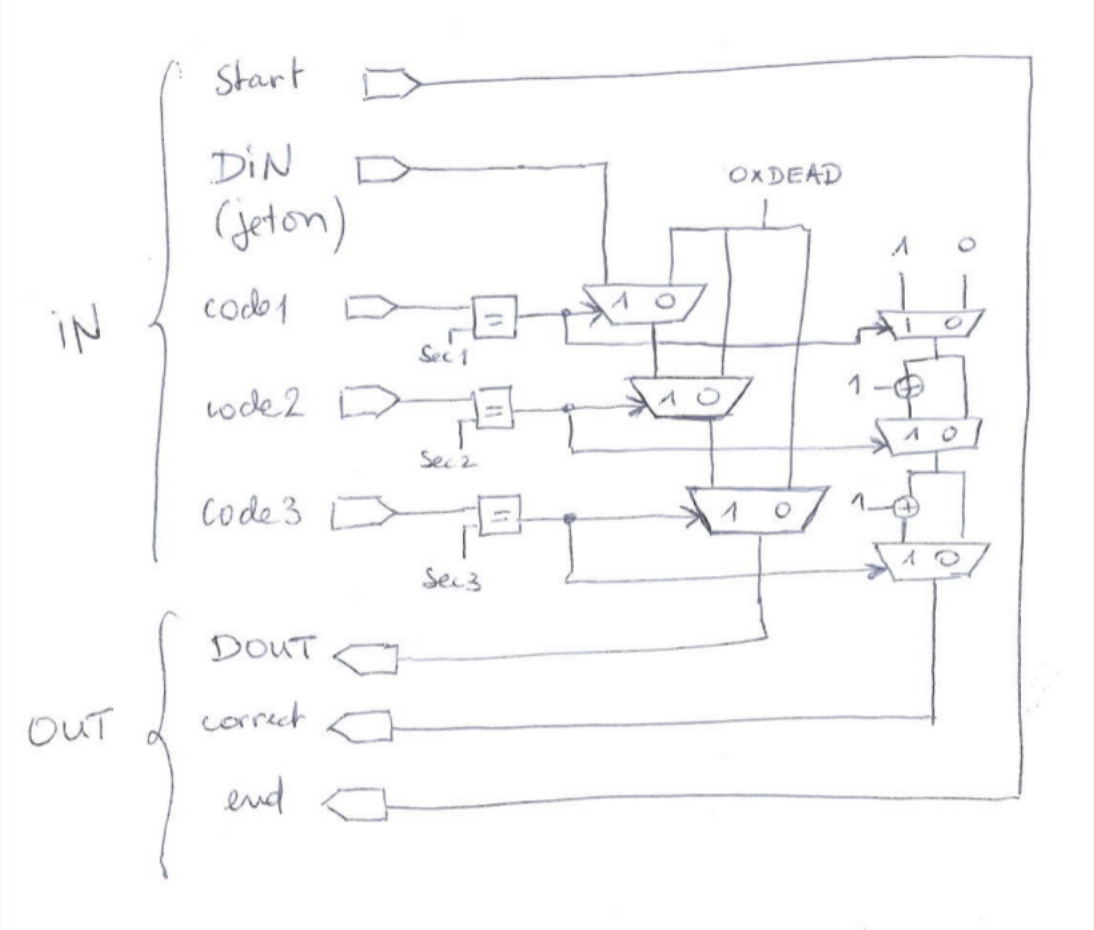
1. Le jeu est inspiré du jeu "Dix de Chute" des années 70.

Solution

1. Architecture :

Le but est de manipuler des circuits pour réaliser un “chemin de données”. On choisit de représenter le jeton par un vecteur de bits de taille $n = 16$ (à cause du code `0xDEAD`).

À chaque niveau, un comparateur vérifie si le code d'entrée correspond au code secret. La sortie du comparateur pilote un multiplexeur : si le code est bon, le jeton actuel passe au niveau suivant ; sinon, la valeur `0xDEAD` est injectée et propagée jusqu'à la sortie. Un compteur peut être incrémenté à chaque comparaison réussie.



2. Calcul de complexité :

Pour 9 niveaux avec des codes de 4 bits : il y a $(2^4)^9 = 2^{36}$ combinaisons à tester.

À 10^6 tests/seconde, le temps au pire cas est :

$$\frac{2^{36}}{10^6} \approx 68\,719 \text{ secondes} \approx \textbf{19} \text{ heures}$$

Pour des codes sur 5 bits : $(2^5)^9 = 2^{45}$ combinaisons.

$$\frac{2^{45}}{10^6} \approx 35\,184\,372 \text{ secondes} \approx \textbf{407} \text{ jours}$$

TD 2 – Électronique Numérique

Logique Séquentielle

Notes et Conseils

L'objectif de ce TD est la découverte des circuits *séquentiels*. Nous commençons par découvrir sa brique fondamentale : la *bascule D*. Elle permet de mémoriser des signaux à des instants discrets. A travers différents exercices (dont l'un de *reverse engineering*), nous cherchons à appréhender la manière de concevoir différents circuits séquentiels : ce temps discret nécessite de penser en terme de *suites numériques*, en raisonnant sur l'instant présent et l'instant passé.

Première partie

Bascule D et circuits séquentiels élémentaires

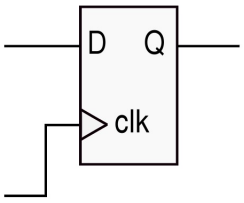
1 Découverte de la bascule D

Le paradigme dominant en conception de circuits numérique est le paradigme *synchrone* et *mono-horloge* : l'ingénieur organise sa réflexion et sa méthodologie de conception en reposant sur l'idée de la délivrance d'un *tempo* externe, parfait et régulier. Dans les faits, cet *tempo* est délivré par un signal carré d'horloge. Il existe certes des difficultés techniques à cette délivrance (*clock skew* notamment), mais le principe de *séparation des préoccupations* lui permet de les négliger.

Le circuit élémentaire sensible à l'horloge, et qui va permettre de concevoir des circuits séquentiels est la **bascule D**. C'est la *seule bascule qui va nous préoccuper ici*. C'est l'élément qui va nous permettre de distinguer *l'avant* de *l'après*. Dans votre esprit, il ne doit pas y avoir d'autre.

Toutefois, tout comme en Physique l'*atome* n'est finalement pas la brique *insécable* qu'on croyait, la bascule D possède bien évidemment une constitution interne : elle est souvent présentée comme le résultat d'un assemblage savant autour d'une autre bascule *asynchrone* cette fois, appelé *latch (verrou) RS*. En réalité, il existe beaucoup d'autres architectures VLSI (intégration à très grande échelle), qui utilisent des principes physiques différents pour mémoriser l'état. Nous prenons l'option pédagogique de nous concentrer uniquement sur la bascule D ici.

Le symbole de la bascule D, ainsi que sa table de vérité, sont rappelés ci-dessous.



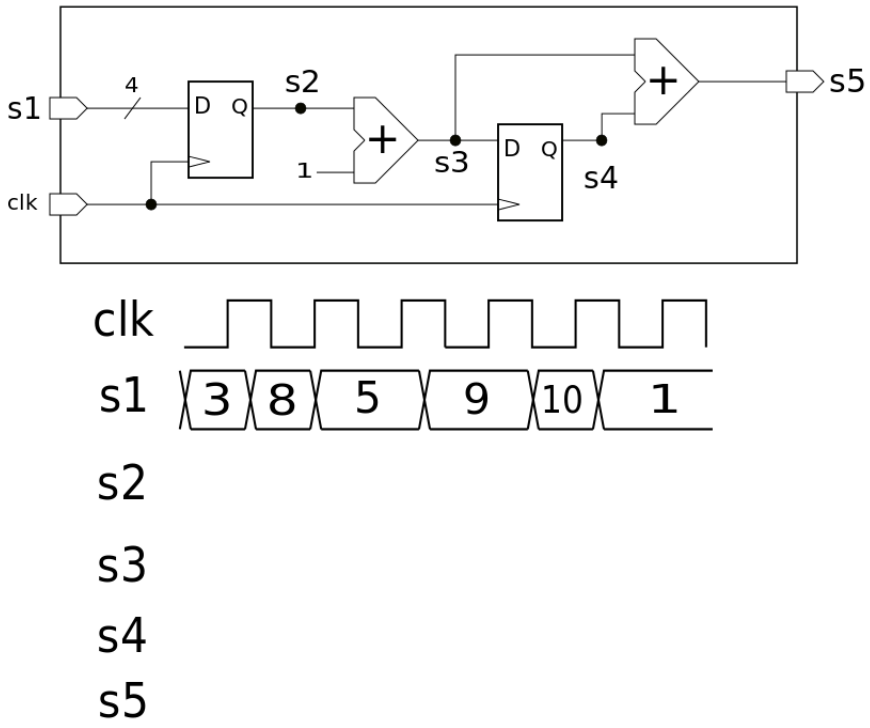
D	clk	Q_{n+1}
0	⎍	0
1	⎍	1
X	0 / 1	Q_n

La bascule D a pour fonction de recopier l'entrée D sur sa sortie Q, lors d'un court temps d'échantillonnage : il s'agit généralement du **front montant** de l'horloge. Cette horloge est supposée périodique. Pour que l'échantillonnage se passe correctement *d'un point de vue physique*, le signal d'entrée doit respecter deux contraintes : il doit être parfaitement stable *un peu avant* et *un peu après* le front montant. Ces instants s'appellent temps de *setup* et temps de *hold*. La donnée est électriquement recopiée après un instant également très court appelé "clock to Q". Tous ces temps sont très inférieurs à la période d'horloge.

La bascule D permet ainsi de travailler sur un signal stable, pendant toute une période : le temps est désormais **discrétisé**. Il n'y a plus à se soucier des temps de propagation caractéristiques – et parfois fluctuants – des différentes portes logiques réalisant les calculs ! Il suffit d'attendre une période d'horloge suffisamment longue pour observer un résultat combinatoire correct, échantillonné dans une bascule. L'horloge absorbe ces variations : c'est l'*hypothèse synchrone*. L'écoulement du temps ne se fait que par "quantum" de temps : celui des "tops" (ou "ticks") de cette horloge périodique.

Il est même possible d'abstraire le fonctionnement du circuit encore plus en supposant que seuls ces instants d'échantillonnage sont significatifs et que les calculs s'effectuent instantanément sur ces "tops" (on parle de calcul en "temps zéro"), et que rien ne se passe entre ces "tops" ! Mais cela reste "une vue de l'esprit" : c'est un des nombreux mécanismes d'*abstraction* rencontrés en conception de circuits numériques.

Exercice : Compléter le chronogramme suivant où un signal S_1 sur 4 bits, extérieur au système et asynchrone, est fourni comme entrée (sous sa forme entière).



On précise ici que :

— nous ne tenons pas compte de la notion de temps de *setup* et temps de *hold*, ni du temps $t_{clk \to Q}$;

— les zones de transition des 4 bits se présentent ici comme des “X” ;

— pour les signaux internes au système, on ne représentera pas ces transitions, supposées idéales : seuls des rectangles suffiront ;

— on suppose que les sorties Q sont initialisées à 0.

Solution

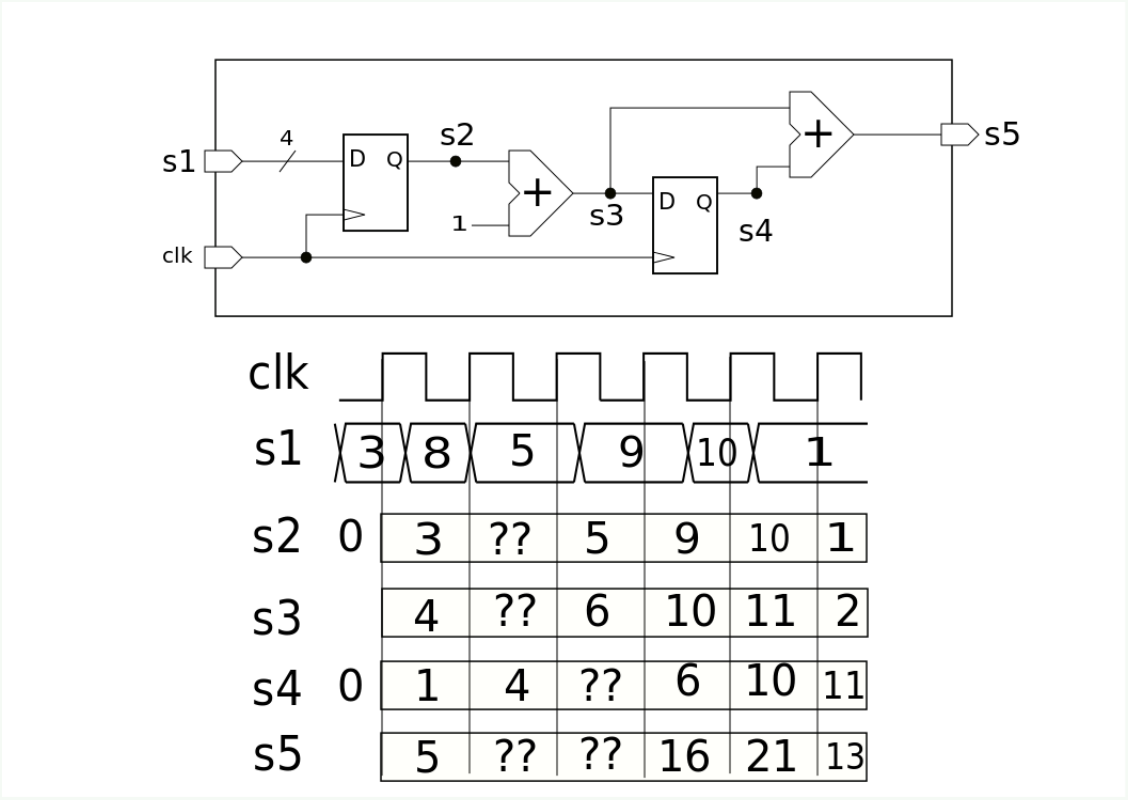


FIGURE 1 – Chronogramme complété

Les plus observateurs noteront que le signal S_5 possède deux valeurs 16 et 21 qui ne peuvent pas être encodées sur 4 bits (il en faut 5). Toutefois, dans cet exercice, rien ne stipule que tous les signaux doivent forcément être encodés sur 4 bits. Malgré tout, si on impose cette contrainte de 4 bits sur tous les signaux, ces deux valeurs seront tronquées : 16 sera tronqué à 0 et 21 sera tronqué à 5. Dans les deux cas, le MSB à 1 en position 5 est perdu.

Une autre question récurrente sur cet exercice est la suivante : on a expliqué que l'échantillonnage de S_1 ne peut pas bien se passer sur la valeur incertaine entre 8 et 5. Le déclenchement de la recopie, par la bascule, sur front montant intervient sur un signal instable et dont la valeur ne peut être connue avec certitude. Pourtant, la solution proposée à l'exercice conduit à dessiner de tels cas, partout dans le tableau : les transitions entre les valeurs sont inscrites sur le front lui-même, ce qui a de quoi perturber...

En réalité, la bascule D possède deux fonctions :

— vis-à-vis d'un signal externe (comme S_1), non synchronisé sur l'horloge, la bascule permettra de filtrer le signal, en ne conservant que les valeurs se prêtant à un échantillonnage correct (respect des temps de setup et de hold) ;

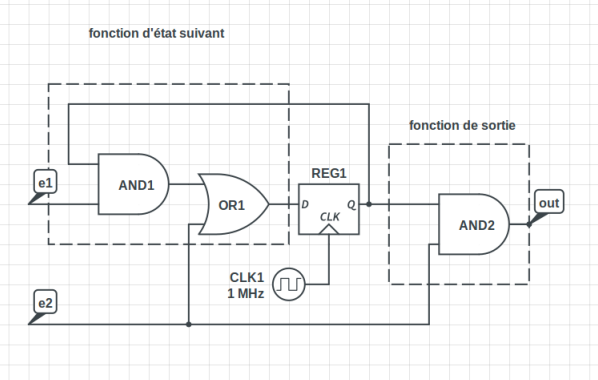
— ensuite, en interne, la bascule D réalise la simple recopie de l'entrée, décalée d'un cycle d'horloge.

Une autre manière de voir les choses, et qui permet d'éviter de considérer deux fonctions distinctes à la bascule, est d'insérer des temps δt_i infinitésimaux entre l'instant précis du front montant et l'apparition de la valeur sur Q_i. Physiquement, ces temps existent (clk-to-Q). Toutefois, il est préférable de ne plus les considérer (on les fait tendre vers 0) : cela permet de simplifier les raisonnements, en retombant dans le monde des suites numériques, plutôt que de conserver ces grandeurs temporelles physiques, par ailleurs compliquées à dessiner !

Il s'agit d'une abstraction, qui fonctionne très bien dans le cas des circuits numériques synchrones. Pour rappel, une abstraction cherche à gommer les détails inutiles (ici le temps physique), tout en conservant les propriétés essentielles (ici la recopie) de l'objet considéré (la bascule).

2 Du circuit au chronogramme

Soit le circuit logique suivant.



Sachant que la bascule D est initialisée à 0, trouver le chronogramme de la sortie *out* correspondante, lorsque les valeurs d'entrée sont (respectivement, à chaque coup d'horloge) :

```
e1 : 0 1 0 1 1 0 1
e2 : 1 1 1 0 1 0 0
```

Solution

On commence par le cycle 1 : on connaît la valeur de $e_1 = 0$ à ce cycle, ainsi que la valeur $Q = 0$ (comme indiqué dans l'énoncé). On en déduit : la sortie du ET vaut 0. D'où, connaissant $e_2 = 1$, on trouve la sortie du OU égale à 1. Cette sortie du OU est également l'entrée D de la bascule. Enfin, pour ce premier cycle, on détermine la sortie OUT, qui vaut 0.
On peut donc passer au cycle 2 : pour cela il faut commencer par répercuter la valeur précédente (1) sur D vers la sortie Q courante de la bascule. Ainsi, Q, au cycle 2, vaut 1. À partir de là, on peut continuer à propager les calculs, de proche en proche.
Un calcul combinatoire impose de rester dans l'instant courant, alors qu'un élément séquentiel (bascule D) impose de se déplacer dans le temps, discrétisé.
On trouve le chronogramme complet :

Cycle	1	2	3	4	5	6	7
e_1	0	1	0	1	1	0	1
e_2	1	1	1	0	1	0	0
AND	0	1	0	1	1	0	0
OR	1	1	1	1	1	0	0
D	1	1	1	1	1	0	0
Q	0	1	1	1	1	1	0
OUT	0	1	1	0	1	0	0

3 Du chronogramme au circuit

Nous proposons ici un exercice plus rare, uniquement par jeu : il s'agit de retrouver le circuit qui a généré les chronogrammes suivants. Il y a probablement plusieurs solutions ! Soyez

perspicaces ! Les signaux e_i représentent des entrées, les signaux s_i des sorties, et les signaux n_i des signaux internes.

```
e1 : 0 0 1 1 0 0 0 0 1 1 0 0 1 0 0 0 0
e2 : 0 0 0 0 1 0 0 1 0 1 1 0 1 1 0 0 0
n1 : 0 0 1 1 1 0 0 1 1 1 1 0 1 1 0 0 0
s1 : 0 0 1 0 0 0 0 1 0 0 0 0 1 0 0 0 0
s2 : 1 1 0 1 1 1 1 0 1 1 1 1 0 1 1 1 1
s3 : ? 1 1 0 1 1 1 1 0 1 1 1 1 0 1 1 1
```

Solution

On voit que :

- n_1 est le OU entre e_1 et e_2 ;
- s_1 marque le début de la séquence des 1 de n_1 : $s_1 = n_{1,t} \cdot \overline{n_{1,t-1}}$ (front montant) ;
- $s_{2,t}$ est l'inverse de $s_{1,t}$;
- s_3 est égal au signal s_2, décalé d'un cycle : $s_{3,t} = s_{2,t-1}$.

Un circuit possible est alors :

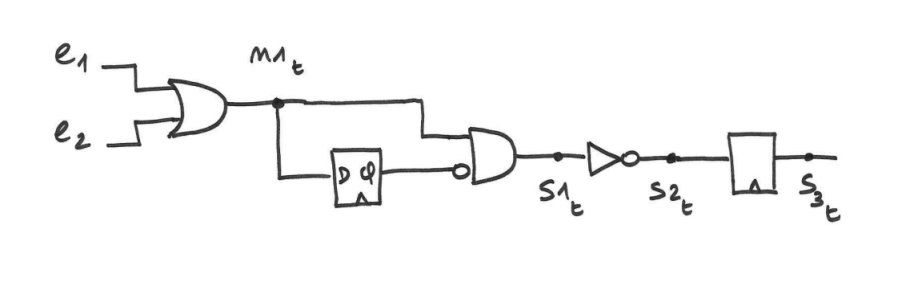


FIGURE 2 – Circuit proposé pour reproduire les chronogrammes

4 Suite de Fibonacci en circuit

La suite de Fibonacci est bien connue : $u_{n+2} = u_{n+1} + u_n$, où $u_1 = 1$ et $u_0 = 0$. Elle date de 1202 et décrit la croissance d'une population de lapins, à partir d'un seul couple. Pour la petite histoire, cette suite a par ailleurs un lien étonnant avec le nombre d'or $\phi = \frac{1+\sqrt{5}}{2} \approx 1{,}618$. En effet, Euler a démontré que pour un n donné (un "step"), on a :

$$u_n = \frac{1}{\sqrt{5}} \left(\phi^n - \phi'^n\right) \quad \text{où} \quad \phi' = -\frac{1}{\phi}$$

1. Dessiner le circuit numérique émettant la suite de Fibonacci. Un changement d'indice simple, au préalable, est conseillé.
2. Chaque bascule D possède un reset actif haut, et un set actif bas. Compléter le schéma précédent, de manière à initialiser correctement les bascules. On dispose d'un bouton poussoir d'initialisation qui fournit un signal de *reset* actif *haut* : lorsqu'on appuie sur ce bouton pour initialiser le circuit, ce signal vaut "1".

Solution

1. On fait le changement d'indice suggéré : $u_n = u_{n-1} + u_{n-2}$. Le décalage temporel correspond à une bascule D. On a donc le schéma ci-dessous. On a omis le signal d'horloge, supposé présent de manière implicite.

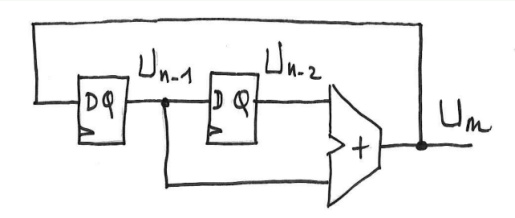


FIGURE 3 – Circuit générateur de la suite de Fibonacci

2. L'initialisation se fait comme présenté sur le schéma ci-dessous.

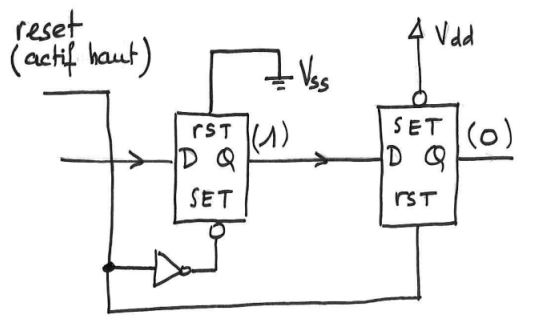


FIGURE 4 – Circuit avec initialisation par reset/set

Deuxième partie

Chemin de données ou *datapath*

Nous avons déjà abordé la notion de "chemin de données" dans le "jeu des cambrioleurs" (TD1) : il s'agissait uniquement d'acheminer, en combinatoire, une donnée d'une entrée vers une sortie, sans traitement spécifique. En général, les *datapaths* sont plus complexes et associent à la fois logique séquentielle et logique combinatoire. Ils constituent le coeur des architectures de calculs.

L'essentiel de leur puissance de calcul provient de deux notions fondamentales :

— **Le parallélisme** : il s'agit d'effectuer plusieurs opérations durant un seul cycle d'horloge, grâce à l'utilisation de plusieurs opérateurs matériels. Pour rappel, dans un processeur classique (et comme dans les langages dits "séquentiels"), les opérations se font en séquence, les unes à la suite des autres. Ici, à l'inverse, nous sommes en mesure de construire des calculateurs qui effectuent plusieurs opérations simultanément.

— **Le pipeline** : il s'agit d'effectuer des opérations "à la chaîne", sur des données, chaque *étage* du pipeline ayant une action spécifique sur ces données.

Nous allons d'abord chercher à illustrer le rôle de la bascule D dans la structure d'un pipeline élémentaire : le registre à décalage.

5 Pipeline, Registres à décalage et Applications

Le registre à décalage le plus simple consiste à relier la sortie d'une première bascule D à l'entrée d'une seconde bascule D. Ceci se généralise à un nombre L de bascules ainsi enchaînées

les unes aux autres (sans rebouclage). Une donnée e qui se présente au cycle n mettra L cycles (ou "coups d'horloge") à parvenir à la sortie s.

Sa fonction est double :

1. Retarder la donnée a_0 d'un temps discret L : elle se présente sur l'entrée e au temps t et ressort sur la sortie s du dispositif au temps $t + L$.
2. Autoriser une donnée a_1 à se présenter sur l'entrée e au temps $t + 1$, en même temps que a_0 est échantillonnée par la seconde bascule, etc.

Exercice 1 : registre à décalage simple

Dessiner un registre à décalage de taille 4. Nommer correctement ses signaux et écrire l'équation de la sortie $s(t)$.

Solution

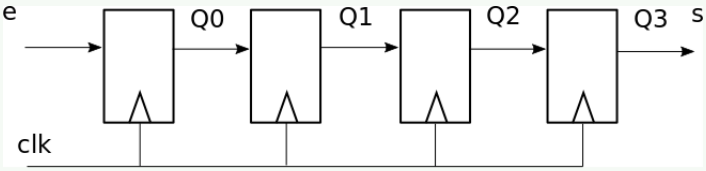


FIGURE 5 – Registre à décalage de profondeur 4.

L'équation de sortie est :

$$s(t) = e(t - 4)$$

Le chronogramme est le suivant :

Cycle	0	1	2	3	4	5	...
e	a_0	a_1	a_2	a_3	a_4	a_5	...
s	?	?	?	?	a_0	a_1	...

Exercice 2 : registre à décalage commandé

Modifier le registre à décalage précédent de manière à le commander par un signal `push`, qui autorise les données à s'acheminer progressivement vers la sortie lorsque ce signal vaut "1".

Solution

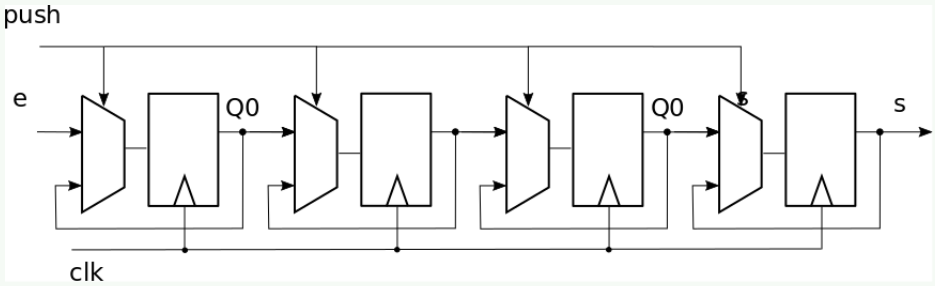


FIGURE 6 – Registre à décalage de profondeur 4, avec signal de contrôle `push`.

Exercice 3 : moyenne mobile

Soit un flux séquentiel de données (boursier, médical, etc.), dont les données successives arrivent au rythme de 100 MHz. La moyenne mobile (ou moyenne glissante, ou *sliding average* en anglais) d'un tel flux continu permet de lisser une moyenne au cours du temps.

1. Calculer rapidement la moyenne mobile, sur 4 échantillons, du flux suivant :

$$\ldots, 0, 0, 0, 0, 8, 9, 13, 4, 3, 2, 1, 0, 0, 0, \ldots$$

2. Concevoir un circuit numérique qui réalise le calcul d'une moyenne mobile sur 4 échantillons de nombres entiers, supposés non signés. Proposer une seconde solution. Laquelle est meilleure ? On s'autorise à utiliser la notion d'additionneur, mais pas de diviseur.
3. Combien de bits sont nécessaires pour calculer la somme de 4 données codées sur 8 bits ? Combien pour la moyenne mobile ?
4. Combien de bits sont nécessaires pour calculer la somme de n données codées sur m bits ?
5. Sachant que le temps de traversée de l'additionneur est 4 ns, quelle est la fréquence atteignable du circuit ? Est-ce compatible avec la fréquence d'arrivée des données ?

Solution

1. La moyenne donne :

$$\ldots, ?, ?, ?, 0, 2, 4, 7, 8, 7, 5, 2, 2, 0, 0, \ldots$$

2. Une solution consiste à réaliser les additions en parallèle. Une seconde à les chaîner. Il n'y a pas de meilleure solution, tant que la fréquence du circuit est atteinte. Il faut pour cela calculer le chemin critique dans les deux cas.
3. On se convainc assez facilement que pour 4 données de 8 bits, il faut **10 bits**. Pour la moyenne mobile on est ramené à **8 bits**.
4. Soit $D(x)$ le nombre de bits nécessaire pour coder x (entier positif non signé). On a $D(x) = \lfloor \log_2(x) + 1 \rfloor$. Pour l'addition S de n données codées sur m bits, on a donc :

$$D(S) = \lfloor \log_2(n \cdot (2^m - 1)) + 1 \rfloor$$

5. $4 + 4 = 8$ ns. On inverse : la fréquence atteignable est de **125 MHz** au maximum. C'est parfaitement compatible avec la fréquence de 100 MHz : il faudra veiller à ce que le circuit fonctionne exactement à la même fréquence que l'émetteur des données.

Exercice 4 : look-up table de FPGA

Les FPGA sont des circuits réguliers qui ont l'étonnante propriété d'être reprogrammables à volonté. On parle de "circuits reconfigurables". Les FPGA sont constitués d'un assemblage de briques élémentaires appelées BLE (*Basic Logic Element*), interconnectées en réseau. Leur topologie détaillée est présentée dans le polycopié du cours.

Un BLE se compose notamment d'une LUT (*look-up table*) à n entrées : c'est une mémoire qui permet d'enregistrer les valeurs attendues de n'*importe quelle fonction booléenne à n entrées*. C'est l'équivalent matériel d'une petite table de vérité, qui se présente sous la forme d'un registre à décalage *avec* `enable` (voir exercice précédent où on a appelé ce signal `push`) [1] à 2^n bascules : le flux qui entre dans ce registre à décalage s'appelle un *bitstream*. Il agit comme une programmation ou *configuration* de la LUT.

1. ...pour les besoins de notre exercice. Les LUTs des FPGAs sont réalisées au niveau transistors.

Chaque sortie Q_i des bascules de la LUT est connectée à une des 2^n entrées d'un multiplexeur. Les signaux de contrôle du multiplexeur sont les n entrées, notées ici $\{a, b, c, \ldots, \alpha_{n-1}\}$.

1. Dessiner une LUT 2.
2. On suppose que le bitstream est entré à l'aide d'un signal de contrôle auxiliaire `push` (cf exercice 2) de la manière suivante : 1000, où le bit le plus à droite représente la première donnée. Calculer la sortie du multiplexeur pour les combinaisons des entrées a, b.
3. Quelle est la fonction réalisée par le dispositif ?
4. Conclure en proposant un bitstream pour la fonction $xor(a, b)$.

Solution

1. Voir figure **??**.

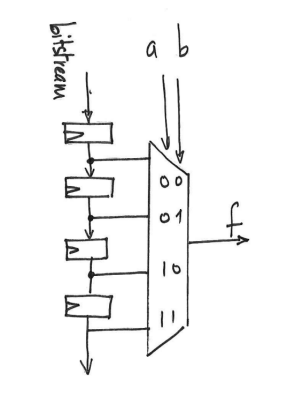


FIGURE 7 – LUT2 (2 entrées) – On a omis le bit `enable` (ou `push`) pour simplifier le schéma, mais ce contrôle est nécessaire afin de bien séparer les phases de programmation (*configuration*) et d'utilisation.

2. Les combinaisons de sortie sont :

a	b	f
0	0	1
0	1	0
1	0	0
1	1	0

3. C'est la fonction **NOR2**. Le bitstream nous a permis de configurer la LUT en NOR2 !
4. Pour un XOR, le bitstream est **0110**.

TD 3 – Électronique Numérique

Automates ou Machines d'états finis

Notes et Conseils

Remarque : Ce TD aborde plusieurs points relatifs aux *automates*, aussi appelés *machines d'états finis*[a] (*finite state machines*, FSM) :

- Diagrammes états-transitions : machines de Mealy et de Moore, et leur équivalence.
- Causalité des machines d'états finis.
- Conception d'un automate matériel à partir d'un diagramme états-transitions.

Les automates peuvent être vus comme l'outil conceptuel équivalent aux équations différentielles du monde analogique et continu : ces automates permettent de décrire l'évolution temporelle d'un système. Ils sont donc fondamentaux, non seulement pour la conception en Electronique, mais plus généralement pour toute l'Informatique.

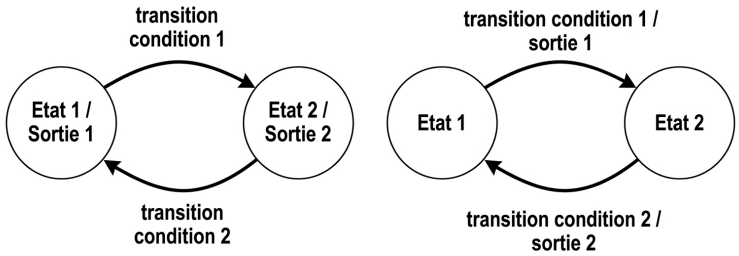
a. Ce nom quelque peu étrange signifie que le *nombre* d'états est *fini*. Il existe également des automates dont le nombre d'états est infini...

1 Diagramme états-transitions : Moore et Mealy

On rappelle que le diagramme *états-transitions* (ou *state transition diagram*, aussi appelé *diagramme à bulles*) est une représentation graphique abstraite du comportement d'un automate. Cette représentation consiste à décrire l'enchaînement des transitions d'un état à l'autre par une flèche, les états étant représentés par des cercles ou *bulles*.

On distingue deux grands types de FSM . Dans le cas des automates de Moore, les sorties ne dépendent que des états, alors que dans le cas des automates de Mealy, les sorties dépendent à la fois des états et des entrées.

Sur les transitions, on ajoute les *conditions* booléennes $c_i(s)$ qui autorisent à passer d'un état à un autre, ainsi que d'éventuelles sorties (généralement après une ou deux barres verticales) dans le cas des automates de Mealy. Lorsqu'on a affaire à un automate de Moore, les sorties sont annotées en conséquence dans les bulles (ou juste à côté).



(a) FSM de Moore et Mealy. Attention ! Les deux automates présentés ici ne sont ici pas équivalents en terme de comportement ! Voir exercice)

2 Causalité d'une FSM

Pour qu'une FSM soit considérée comme *causale*, correcte ou *consistante*, elle doit vérifier les deux propriétés suivantes :

— **Réactivité** (exhaustivité) : il doit exister une condition vraie sur au moins une des transitions sortant d'un état.

$$\forall s \in S : \sum_i c_i(s) = 1$$

— **Déterminisme** (exclusivité) : deux conditions sur les transitions sortant d'un état ne peuvent être vraies en même temps.

$$\forall s \in S, \forall (i,j) \text{ avec } i \neq j, \quad c_i(s) \cdot c_j(s) = 0$$

Une autre manière de résumer ces deux propriétés est qu'il existe – à tout instant discret – *un, et un seul, état suivant*. Notons qu'un état peut avoir comme état suivant lui-même : dans ce cas, le système décrit ne change pas d'état.

Questions

1. Observez la figure 1.Vérifier la consistance de la machine d'état suivante.
2. Est-ce un automate de Moore ou de Mealy ?

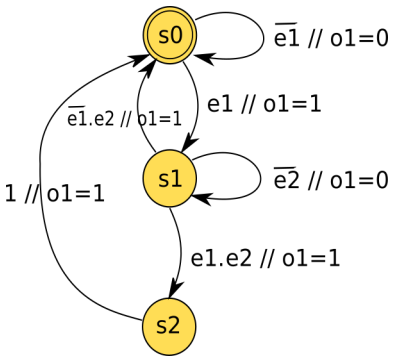


FIGURE 1 – FSM de l'exercice 1 et 2

Solution

Il y a plusieurs manières de procéder. Par exemple, on calcule les formules :

— **Exhaustivité :**
  — Pour S0 : $\overline{e_1} + e_1 = 1$
  — Pour S1 : $\overline{e_2} + \overline{e_1}e_2 + e_1e_2 = 1$
  — Pour S2 : $1 = 1$

— **Exclusivité :**
  — Pour S0 : $\overline{e_1} \cdot e_1 = 0$
  — Pour S1 : $\overline{e_2}\overline{e_1}e_2 + \overline{e_2}e_1e_2 + \overline{e_1}e_2e_1e_2 = 0 + 0 + 0 = 0$
  — Pour S2 : une seule transition. Il n'y a pas de problème d'exclusivité.

C'est un automate de **Mealy**, car les sorties dépendent des transitions, et non pas seulement des états.

3 Du diagramme états-transitions au circuit numérique.

Nous cherchons ici à réaliser l'électronique de l'automate de l'exercice précédent.

Notes et Conseils

La méthode manuelle consiste à décrire tout d'abord dans un tableau :
- L'évolution des états en fonction des entrées, en conservant leur nom *symbolique*.
- L'évolution des sorties en fonctions des états, et des entrées (cas de Mealy).

Après **choix d'un encodage binaire des états**, on connaît le nombre n de bascules D impliquées :
- On réécrit le tableau précédent pour chacune des bascules D_i.
- On déduit les équations $D_i = f(Q_0, Q_1, \ldots, Q_{n-1}, e_0, \ldots)$, où Q_j représente l'état courant de la bascule j et $\{e_k\}$ l'ensemble des entrées du circuit.

On rappelle qu'il existe un nombre infini de possibilité d'encodage des états. Cet encodage influe sur le nombre de bascules D nécessaires au stockage des états, mais également sur la complexité de la logique combinatoire de la fonction de transitions et de la fonction de sortie (et donc de la rapidité du circuit).

On s'appuie sur les tableaux génériques suivants 1 et 2, qui permettent de procéder de manière systématique.

TABLE 1 – Encodage symbolique (nom des états préservés)

état courant	entrée 1	...	entrée n	état suivant	sortie 1	...	sortie n

TABLE 2 – Encodage binaire (états encodés –ici sur 2 bits)

Q1	Q0	entrée 1	...	entrée n	D1	D0	sortie 1	...	sortie n

Questions

- Etablir le tableau d'évolution des états symboliques et de la sortie.
- Choisir l'encodage *dense* le plus naturel.
- Etablir le tableau d'évolution des états encodés et de la sortie.
- Etablir les équations de chacun des bits d'états.
- Etablir les équations de chacun des bits de sortie.
- Dessiner le circuit à l'aide portes logiques élémentaires.

Solution

État courant	e_1	e_2	**État suivant**	o_1
S0	0	X	S0	0
S0	1	X	S1	1
S1	X	0	S1	0
S1	0	1	S0	1
S1	1	1	S2	1
S2	X	X	S0	1

Puis encodons les états symboliques S0, S1 et S2 par 00, 01, 10 :

Q_1	Q_0	e_1	e_2	D_1	D_0	o_1
0	0	0	X	0	0	0
0	0	1	X	0	1	1
0	1	X	0	0	1	0
0	1	0	1	0	0	1
0	1	1	1	1	0	1
1	0	X	X	0	0	1

Attention : ne pas chercher les équations directement à partir de ce tableau ! Il manque des combinaisons. Utilisons plutôt le tableau de Karnaugh réorganisé :

	$\overline{e_1}\overline{e_2}$	$\overline{e_1}e_2$	e_1e_2	$e_1\overline{e_2}$
$\overline{Q_1}\overline{Q_0}$	00	00	01	01
$\overline{Q_1}Q_0$	01	00	10	01
Q_1Q_0	xx	xx	xx	xx
$Q_1\overline{Q_0}$	00	00	00	00

À partir de ce dernier tableau, on peut déterminer les équations de D_1, D_0 et o_1 :

$$\begin{aligned} D_1 &= Q_0 \cdot e_1 \cdot e_2 \\ D_0 &= \overline{Q_1}\overline{Q_0}e_1 + Q_0\overline{e_2} \\ o_1 &= \overline{Q_1}\overline{Q_0}e_1 + Q_0e_2 + Q_1 \end{aligned}$$

4 Automate *détecteur de séquence*

Il est fréquent de devoir détecter une séquence particulière dans un flux de données. Les automates se prêtent très bien à ce genre de détection. On peut citer :

— L'analyse de paquets réseaux, détection de paquets interdits (entrants ou sortants) dont on connaît une certaine signature numérique (travail de filtrage d'un proxy, etc.).

— L'analyse de flux multimédia : par exemple *start codes* de séquences, dans un film (stocké en numérique, dans un format de compression).

— L'analyse génomique : il existe des accélérateurs FPGA dédiés à la reconnaissance de séquence du génome, etc.

4.1 Détecteur de séquence `10` : Moore versus Mealy

Dans le cadre de la conception d'un sous-système de communication série, on souhaite réaliser un détecteur de séquence capable de repérer le motif binaire **"10"** sur un flux de données synchronisé par une horloge.

Le système possède :

— Une entrée binaire synchrone X (le bit courant),

— Une sortie binaire synchrone Z qui vaut **1** si et seulement si la séquence **"10"** vient d'être détectée sur l'entrée X, et **0** sinon.

On considère que les séquences peuvent se chevaucher (par exemple, dans la séquence `1 1 0`, le motif **"10"** est détecté sur les deux derniers bits).

1. **Automate de Mealy.** Concevoir un automate de type **Mealy** réalisant cette fonction. On donnera :
   — la description des états et leur signification,
   — la table des transitions (format : *état suivant / sortie*).
2. **Automate de Moore.** Concevoir un automate de type **Moore** ayant un comportement fonctionnel équivalent. On donnera :
   — la description des états, leur signification et la sortie associée,
   — la table des transitions.
3. **Comparaison temporelle.** Relever les valeurs de la sortie Z pour les deux automates sur la séquence d'entrée suivante, appliquée à chaque front montant de l'horloge :

   $$X = 0,\ 1,\ 0,\ 0,\ 1,\ 1,\ 0$$

   On supposera que les deux automates sont initialisés dans leur état de départ avant le premier front.
4. **Analyse.** Les deux automates sont-ils strictement équivalents ? Justifier en précisant la différence fondamentale entre les machines de Moore et de Mealy, et expliquer le phénomène observé à la question 3.

Solution

1. Automate de Mealy.
Dans un automate de Mealy, la sortie dépend de l'état *présent* et de l'entrée *présente*. La réaction est donc immédiate (combinatoire).

— S_0 : état initial (aucun `1` en attente, ou dernier bit reçu = `0`).
— S_1 : on vient de recevoir un `1` (le `0` suivant complètera le motif).

État présent	**Si** $X = 0$	**Si** $X = 1$
$\rightarrow S_0$	S_0 / $Z = 0$	S_1 / $Z = 0$
S_1	S_0 / $\mathbf{Z = 1}$	S_1 / $Z = 0$

2. Automate de Moore.
Dans un automate de Moore, la sortie ne dépend *que* de l'état présent. Il faut donc un état supplémentaire pour « mémoriser » le fait d'avoir détecté la séquence.

— S_0 : état initial, sortie $Z = 0$.
— S_1 : on vient de recevoir un `1`, sortie $Z = 0$.
— S_2 : on vient de détecter **"10"**, sortie $\mathbf{Z = 1}$.

État présent	**Sortie** Z	**Si** $X = 0$	**Si** $X = 1$
$\rightarrow S_0$	0	S_0	S_1
S_1	0	S_2	S_1
S_2	1	S_0	S_1

3. Comparaison temporelle.
Séquence d'entrée : $X = 0,\ 1,\ 0,\ 0,\ 1,\ 1,\ 0$.

Cycle	1	2	3	4	5	6	7
Entrée X	0	1	0	0	1	1	0
Sortie Mealy Z	0	0	**1**	0	0	0	**1**
Sortie Moore Z	0	0	0	**1**	0	0	0

4. Analyse.
Les deux automates sont **fonctionnellement équivalents** : ils reconnaissent exactement la même séquence. Cependant, ils ne sont **pas synchrones temporellement**.

— Dans l'automate de **Mealy**, la sortie est produite par un réseau combinatoire : dès que l'entrée X passe à `0` au cycle 3 (complétant le motif `10`), la sortie Z réagit *immédiatement* dans le même cycle.
— Dans l'automate de **Moore**, la transition vers l'état S_2 n'est effective qu'au front d'horloge suivant. La sortie $Z = 1$ n'apparaît donc qu'au cycle 4.

Conclusion : la sortie d'un automate de Moore est **toujours décalée d'un cycle d'horloge** par rapport à celle d'un automate de Mealy équivalent. En conception numérique (FPGA, ASIC), on ajoute généralement une bascule de sortie à l'automate de Mealy pour synchroniser sa sortie sur l'horloge : on élimine ainsi les rebonds combinatoires (*glitches*) et on retrouve un comportement temporel identique à celui de Moore, au prix d'un cycle de latence supplémentaire.

4.2 Détecteur de séquence `0110`

On considère le détecteur de séquence de la figure 2. Son rôle est de détecter les séquences *0-1-1-0* sur son unique entrée x. Lorsque cette séquence est détectée, il émet un *1* sur sa sortie y. Un chronogramme est fourni à titre d'exemple. Notez que dans le flux *0-1-1-0-1-1-0*, la séquence apparaît deux fois !

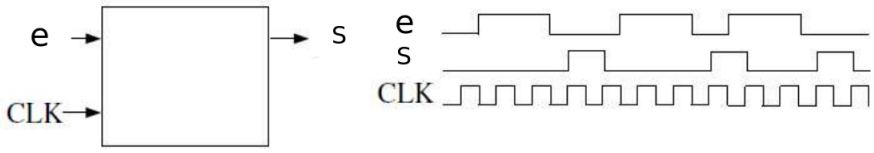


FIGURE 2 – Schéma de principe du détecteur de séquence `0110`. Chronogramme associé.

1. Proposer un diagramme états-transitions pour ce détecteur. On choisira une machine de Moore.
2. Proposer un encodage dense des états.
3. Réaliser le circuit.

Solution

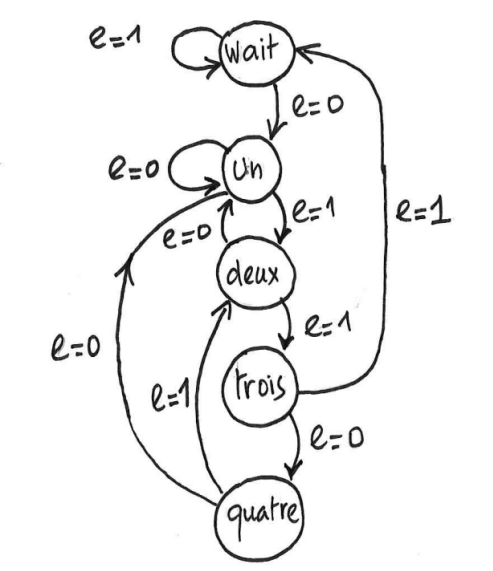


FIGURE 3 – Diagramme états-transitions pour le détecteur de séquences.

L'automate solution est décrit sur la figure 3. On choisit l'encodage dense suivant :

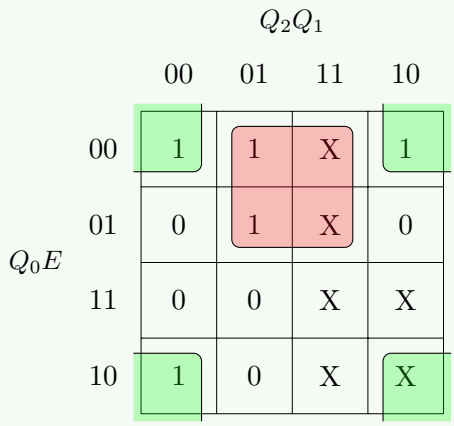
État	**Code**
wait	000
un	001
deux	010
trois	011
quatre	100

On a alors la table de séquencement suivante :

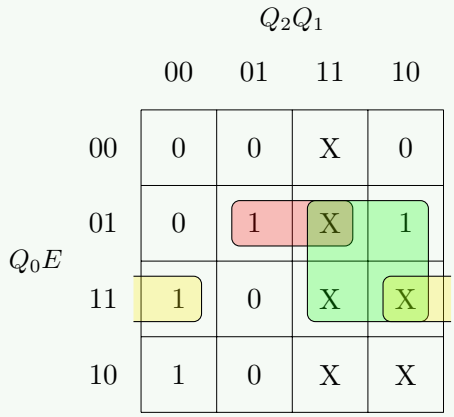
Q_2	Q_1	Q_0	e	D_2	D_1	D_0	s
0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0
0	0	1	0	0	0	1	0
0	0	1	1	0	1	0	0
0	1	0	0	0	0	1	0
0	1	0	1	0	1	1	0
0	1	1	0	1	0	0	1
0	1	1	1	0	0	0	0
1	0	0	0	0	0	1	0
1	0	0	1	0	1	0	0

Solution

Cherchons maintenant les équations, en passant par des tableaux de Karnaugh.



On obtient la formule : $D_0 = Q_1\overline{Q_0} + \overline{Q_1E}$



On obtient la formule : $D_1 = Q_1\overline{Q_0}E + \overline{Q_1}Q_0E +$

Q_2E

Solution

Q_0E \ Q_2Q_1	00	01	11	10
00	0	0	X	0
01	0	0	X	0
11	0	0	X	X
10	0	1	X	X

On obtient la formule : $D_2 = Q_1 Q_0 \overline{E}$

Q_0E \ Q_2Q_1	00	01	11	10
00	0	0	X	1
01	0	0	X	1
11	0	0	X	X
10	0	0	X	X

On obtient la formule : $S = Q_2$ *(Note : l'original indiquait D_3, mais il s'agit de la sortie S ou y).*

5 Arbitre à priorité tournante (round-robin)

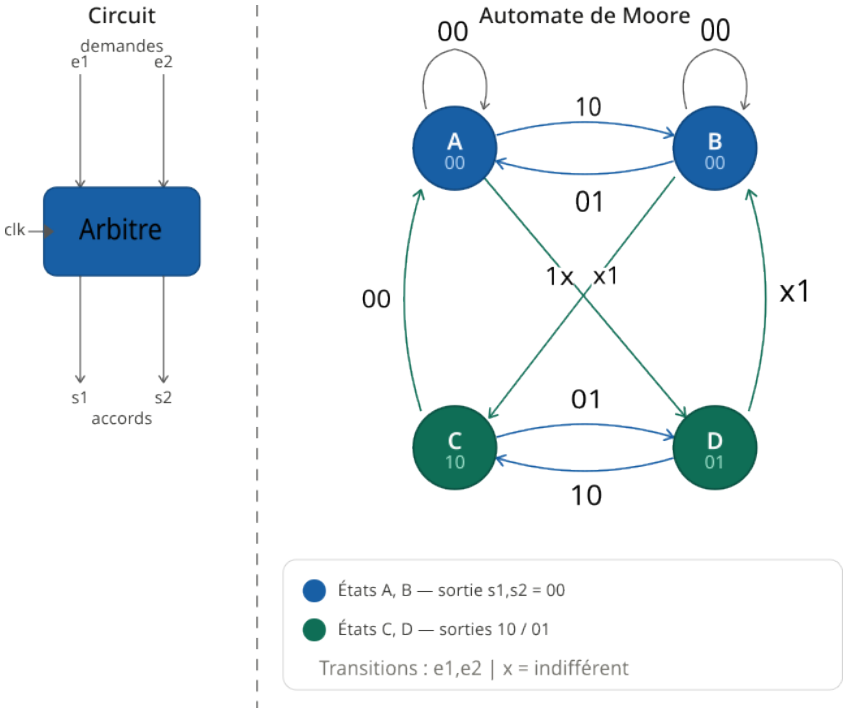
On désire réaliser un circuit d'arbitrage entre deux clients susceptibles d'utiliser une ressource qui n'admet qu'un seul occupant à la fois (par exemple deux enfants qui veulent jouer au vélo, et il n'y a qu'un vélo. Le cas se présente également dans les ordinateurs, où différents périphériques peuvent vouloir accéder à un ensemble de fils –bus central– en même temps). Le but du circuit est d'accorder correctement la ressource à un seul client à la fois.

Les clients manifestent respectivement sur e_1 et e_2 leurs désirs d'utiliser la ressource : $e_i = 1$ si le client i désire la ressource, $e_i = 0$ s'il ne la désire pas.

Le circuit génère sur s_1 et s_2 l'attribution de la ressource à chaque client : $s_i = 1$ si la ressource est attribuée au client i, $s_i = 0$ sinon.

Lorsque la ressource est attribuée à un client, elle le demeure tant que le client la désire.

Lorsque la ressource est libre, elle est attribuée au premier qui la demande ; en cas de demandes simultanées, elle est attribuée selon une certaine priorité, mais pour éviter les injustices,



la priorité entre les clients change à chaque attribution : le client qui obtient la ressource devient le moins prioriatire.

L'analyse du problème conduit à l'automate suivant : il y a quatre états, deux états A et B dans lesquels la ressource est libre, et deux états C et D dans lesquels la ressource est occupée.

Questions

1. Proposer un encodage des états tel que chaque état soit codé sur 2 bits
2. Etablir la fonction de transition du système sous forme de table de vérité. Simplifier.
3. Etablir la fonction de sortie du système. Simplifier.
4. Dessiner le circuit correspondant. Combien de portes faut-il ?

Solution

Cet encodage nous indique qu'il faut 2 bascules D pour stocker l'état du système. Nommons par exemple ces deux bascules Q_1 et Q_0.

Etat	Encodage entier	Encodage binaire	Q_1	Q_0
A	0	00	0	0
B	1	01	0	1
C	2	10	1	0
D	3	11	1	1

Ecrivons la table qui décrit l'évolution des entrées des bascules en fonction des entrées et de l'état courant. Dans le tableau, on a rappelé les états symboliques.

Etat courant			Entrées		Etat futur		
symbole	Q_1	Q_0	e_1	e_2	D_1	D_0	symbole
A	0	0	0	0	0	0	A
A	0	0	0	1	1	1	D
A	0	0	1	0	1	0	C
A	0	0	1	1	1	0	C
B	0	1	0	0	0	1	B
B	0	1	0	1	1	1	D
B	0	1	1	0	1	0	C
B	0	1	1	1	1	1	D
C	1	0	0	0	0	1	B
C	1	0	0	1	1	1	D
C	1	0	1	0	1	0	C
C	1	0	1	1	1	0	C
D	1	1	0	0	0	0	A
D	1	1	0	1	1	1	D
D	1	1	1	0	1	0	C
D	1	1	1	1	1	1	D

partir de ce tableau, on cherche les équations de D_1 et D_0. D_1 se trouve facilement, par une observation attentive du tableau : on peut voir une factorisation aisée :

$$D_1 = e_1 + e_2$$

Solution

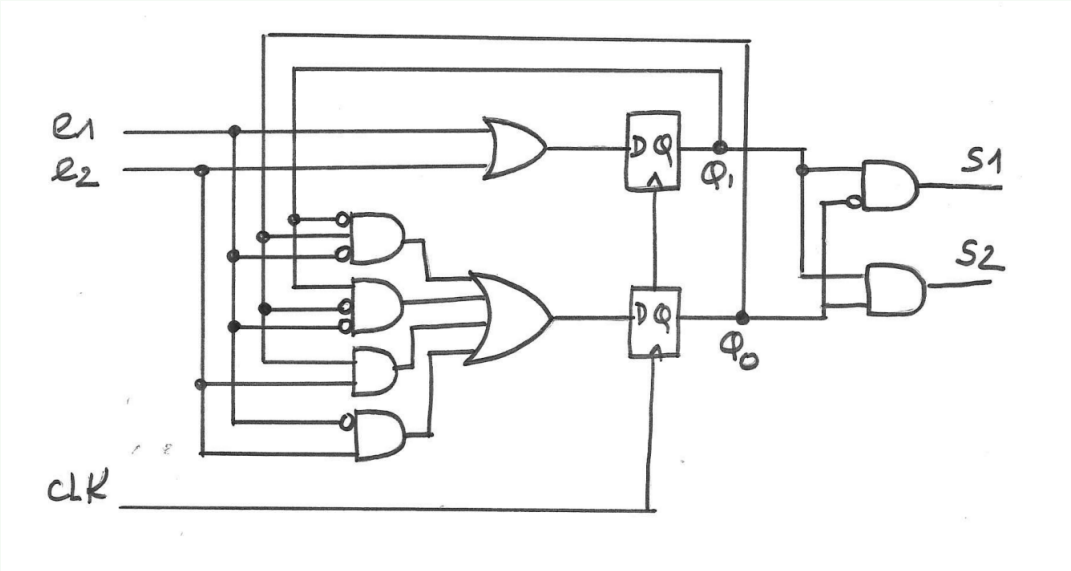
Pour D_0, passons par un tableau de Karnaugh.

$Q_1.Q_0$ \ $e_1.e_2$	00	01	11	10
00		1		
00	1	1	1	
01		1	1	
11	1	1		

Les regroupements amènent à :

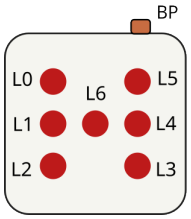
$$D_0 = \overline{e_1}.e_2 + e_2.Q_0 + \overline{e_1}.\overline{Q_1}.Q_0 + Q_1.\overline{Q_0}.\overline{e_1}$$

Calculons les sorties, de la même manière. On trouve : $S_1 = Q_1.\overline{Q_0}$ et $S_2 = Q_1.Q_0$
On peut désormais dessiner le circuit complet :



6 Problème du dé électronique. Encodage des états spécifique.

On doit réaliser une boîte équipée d'un bouton poussoir, de 7 diodes lumineuses (LEDs) d'un circuit logique (à définir) et d'une pile. Ce dispositif doit simuler un dé, lancé aléatoirement : il affiche entre 1 et 6.



Chaque fois que l'utilisateur appuie sur le bouton-poussoir toutes les leds s'allument puis, dès que l'opérateur relâche le bouton, les LEDs se fixent sur l'une des configurations ci-dessous.

A l'appui sur le bouton-poussoir toutes les configurations sont affichées en séquence à la cadence d'une horloge rapide. L'utilisateur ne peut pas distinguer les combinaisons, du fait de la persistence rétinienne. Seule une illumination moyenne lui apparait.

On choisit d'utiliser un encodage des états particulier, *spécifique au problème* : la sortie de chaque bascule $Q_i, i \in 0..6$ pilote directement la led L_i correspondante. La fonction de sortie de l'automate est ainsi la *fonction identité*.

Question

1. Réaliser le tableau décrivant l'évolution de chaque bascule lorsque le dé est sollicité (bouton poussoir appuyé).
2. En déduire les équations constitutives du circuit correspondant.
3. Dessiner le circuit d'une ou deux de ces équations.
4. Rappeler succintement le schéma-blocs composé de 2 blocs : la logique d'état suivant et les bascules D (registres).
5. Choisir un autre encodage (par exemple one-hot) et réaliser à nouveau le circuit.

7 La magie de l'encodage "*one-hot*" ou "*un bit par état*"

Une machine distribue des canettes à 1€. Elle n'accepte que des pièces de 0,50€ et de 1€. Elle ne rend pas la monnaie.

On précise quelques éléments :
- **Entrées :** 'P50' (pièce de 50c) et 'P100' (pièce de 1€). Les deux entrées ne sont jamais actives en même temps.
- **Sorties :** 'DIST' (Distribuer la canette) et 'REND' (Rendre 0,50€ de monnaie - uniquement si le client a donné 1,50€).
- **Fonctionnement :**
  - Si somme totale atteint **1,00€** → 'DIST=1'.
  - Si somme totale atteint **1,50€** → 'DIST=1' ET 'REND=1' (car il a donné 1,50€ pour une canette à 1€).

Questions

1. Déssiner le diagramme états-transitions en conservant le nom des états symboliques.
2. Réaliser l'encodage *one-hot*.
3. Etablir les équations D_i de chacune des bascules mises en jeu.

Solution

Question 1 : Définition des états

États possibles (somme encaissée) :
- 'S0' : 0,00€ (état initial)
- 'S50' : 0,50€
- 'S100' : 1,00€ (on distribue puis on revient à S0)
- 'S150' : 1,50€ (on distribue, on rend 0,50€, puis on revient à S0)

Voici l'ensemble des équations pour les entrées des 4 bascules :

$$\begin{aligned} D_0 &= Q_2 + Q_3 \quad \text{(retour à zéro après distribution)} \\ D_1 &= Q_0 \cdot P50 \\ D_2 &= Q_0 \cdot P100 + Q_1 \cdot P50 \\ D_3 &= Q_1 \cdot P100 + Q_2 \cdot P50 \end{aligned}$$

Remarque : En codage binaire classique, ces équations auraient nécessité des décodeurs complexes. Ici, la logique se limite à des portes ET et OU, ce qui rend la machine plus rapide et plus facile à maintenir.

8 Automate *serrure numérique*. Le problème des *spécifications*.

L'accès à un local est protégé par une serrure codée associée à un automatisme commandant la gâche électrique de la porte. La combinaison secrète est : **A, D, C**.

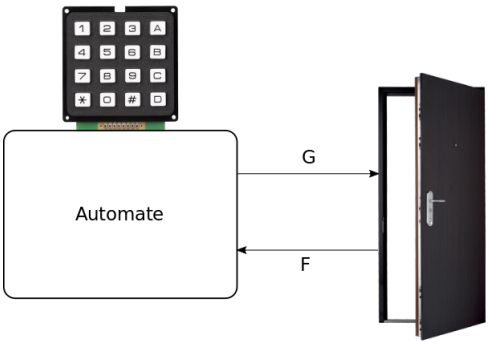


FIGURE 4 – Schéma de principe de la a serrure numérique.

Un capteur permet de connaître l'état de la porte : F indique son état. F vaut 1 si la porte est Fermée, et 0 si elle est ouverte.
G commande l'ouverture de la porte : si $G = 1$ la porte peut être ouverte. Lorsque la porte se referme, on a $G = 0$.
À l'initialisation, la porte est fermée, le contact de porte $F = 1$.
On précise qu'on ne peut appuyer que sur un seul bouton à la fois.

1. Proposer un automate de contrôle du dispositif.
2. Discuter de la robustesse de la solution, et de son réalisme. Proposer des alternatives.
3. Optionnel : construire le circuit complet.

Solution

Dans ce type de dispositifs, les *spécifications* du dispositif sont ambiguës et sujettes à interprétations. Cela est une constante dans ce type de systèmes !

- Par exemple, est-il réaliste de ne pas prendre en compte un temps limite d'attente entre deux actions d'appui sur un bouton ? On pourrait donc, dans un état donné, déclencher un *autre automate* dédié au décomptage du temps (timer).
- Ce timer est-il à déclencher *entre* deux appuis, ou dès qu'on commence la séquence ?
- De même, le sujet stipule que la porte *peut être ouverte* : cela ne signifie pas que le retrait de la gâche entraîne forcément l'ouverture de la porte : on devra donc là aussi attendre une action de l'utilisateur, probablement sur la poignée de la porte.
- S'il y a une poignée d'un côté de la porte, peut-on supposer qu'un *autre* utilisateur puisse ouvrir la porte, de manière inopinée pendant la tentative du premier utilisateur ? Etc.

Nous proposons ici une solution à peu près réaliste : dès que la lettre A est correctement entrée, un timer se déclenche. Si la porte est par ailleurs ouverte (de l'intérieur), le dispositif se remet à l'état d'attente initial.

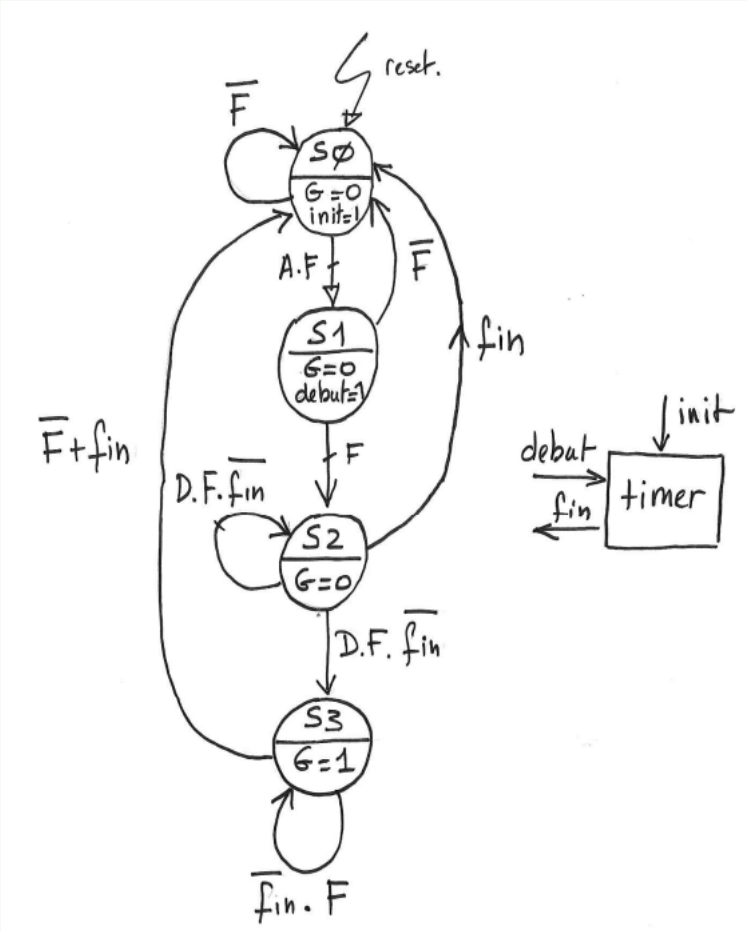


FIGURE 5 – Diagramme état-transition pour la serrure numérique. On fait appel à un autre dispositif numérique : un timer.

TD VHDL – Électronique Numérique

Initiation à VHDL (1/3)

Notes et Conseils

Remarque : Ce TD constitue une introduction à la description matérielle en VHDL. Il est conseillé de travailler les exercices en amont de la séance et de tester les codes sur un simulateur (GHDL) pour valider vos acquis.

1 Premiers pas en VHDL

1. Dessiner l'enveloppe externe du composant décrit par cette entité.

Solution

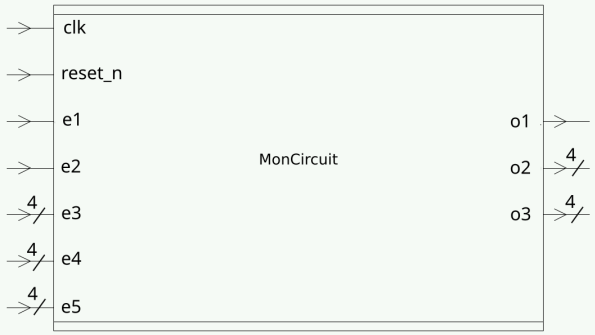


FIGURE 1 – Entity du composant MonCircuit

2. Dessiner l'entité d'un demi-additionneur, puis le code VHDL de cette entité.

Solution

Voir la figure 2.

3. Rappeler la constitution interne du demi-additionneur.

Solution

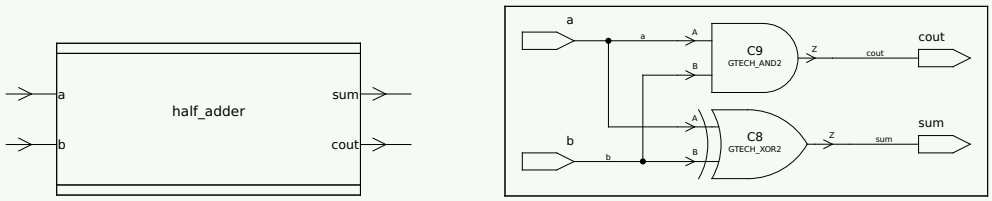


FIGURE 2 – Entity et architecture du demi-additionneur 1 bit

4. Coder l'architecture de ce demi-additionneur.

Solution

```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity half_adder is
  port(
    a,b : in std_logic;
    sum,cout : out std_logic
  );
end half_adder;

architecture logic of half_adder is
begin
  sum <= a xor b;
  cout <= a and b;
end logic;
```

5. Dessinez la constitution interne d'un additionneur 1 bit complet, à partir du demi-additionneur.

Solution

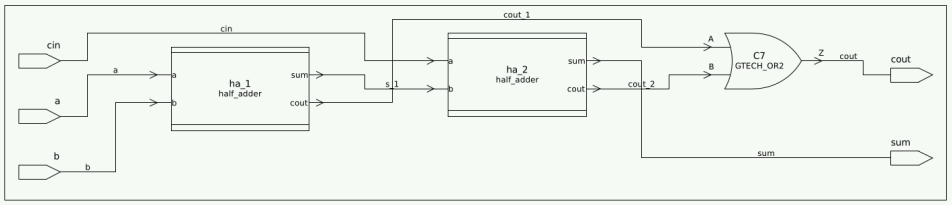


FIGURE 3 – Architecture de l'additionneur 1 bit complet

6. Coder en VHDL l'architecture de cet additionneur 1 bit.

Solution

```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity full_adder is
  port(
    a,b : in std_logic;
    cin : in std_logic;
    sum,cout : out std_logic
  );
end full_adder;

architecture logic of full_adder is
  signal s_1, cout_1,cout_2 : std_logic;
begin

  ha_1 : entity work.half_adder(logic)
  port map(
    a => a,
    b => b,
    sum => s_1,
    cout => cout_1
  );

  ha_2 : entity work.half_adder(logic)
  port map(
    a => cin,
    b => s_1,
    sum => sum,
    cout => cout_2
  );

  cout <= cout_1 or cout_2;

end logic;
```

7. Coder en VHDL l'entité et l'architecture d'un additionneur 8 bits.

Solution

```
library ieee;
use ieee.std_logic_1164.all;

entity adder8b is
  port(
    a,b : in std_logic_vector(7 downto 0);
    sum : out std_logic_vector(7 downto 0);
    cout : out std_logic
  );
end adder8b;

architecture structural of adder8b is
  signal dummy_carry : std_logic;
  signal carry : std_logic_vector(7 downto 0);
begin

  dummy_carry <= '0';

  fa_0 : entity work.full_adder(logic)
  port map(
    a => a(0),
    b => b(0),
    cin => dummy_carry,
    sum => sum(0),
    cout => carry(0)
  );

  fa_1 : entity work.full_adder(logic)
  port map(
    a => a(1),
    b => b(1),
    cin => carry(0),
    sum => sum(1),
    cout => carry(1)
  );

  fa_2 : entity work.full_adder(logic)
  port map(
    a => a(2),
    b => b(2),
    cin => carry(1),
    sum => sum(2),
    cout => carry(2)
  );

  fa_3 : entity work.full_adder(logic)
  port map(
    a => a(3),
    b => b(3),
    cin => carry(2),
    sum => sum(3),
    cout => carry(3)
  );

  fa_4 : entity work.full_adder(logic)
  port map(
    a => a(4),
    b => b(4),
    cin => carry(3),
    sum => sum(4),
    cout => carry(4)
  );
```

2 Le banc de test (testbench)

1. Le testbench présente également une entité. Localisez cette entité. En quoi est-elle particulière ? Comment peut-on l'expliquer ?

Solution

Un tel banc de test possède une entité *vide* : elle ne possède aucune entrée, ni sortie. C'est le laboratoire, portes closes.

2. Un générateur d'horloge est contenu dans le banc de test. Combien de lignes sont nécessaires ? Dessinez le chronogramme de cette horloge.

Solution

Une seule ligne suffit. On inverse le signal tous les demi-périodes d'horloge, à l'aide d'une assignation conditionnée par un signal `running`, qui nous permettra de contrôler l'arrêt de la simulation.

3. Comment notre banc de test provoque-t-il concrètement cet arrêt ?

Solution

Le signal `running` est affecté à `false` à la fin du processus de stimulation. Plus aucun événement n'est créé par le simulateur.

4. Dessiner le chronogramme des stimuli.

Solution

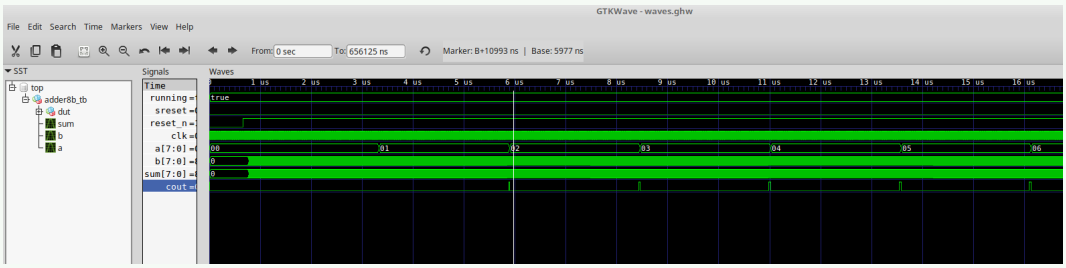


FIGURE 5 – Chronogramme (overview)

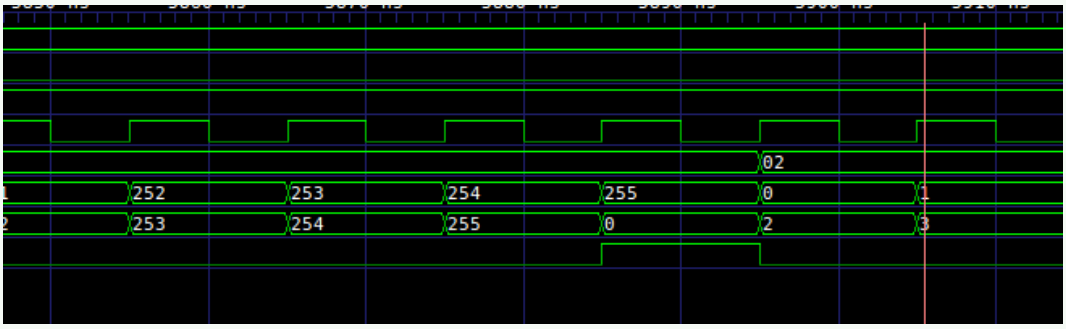


FIGURE 6 – Chronogramme : cas de $1 + 255$

5. L'additionneur fonctionne-t-il avec des nombres signés ?

Solution

Oui ! En tous les cas, un test semble aller en ce sens ! Voir la figure 7.

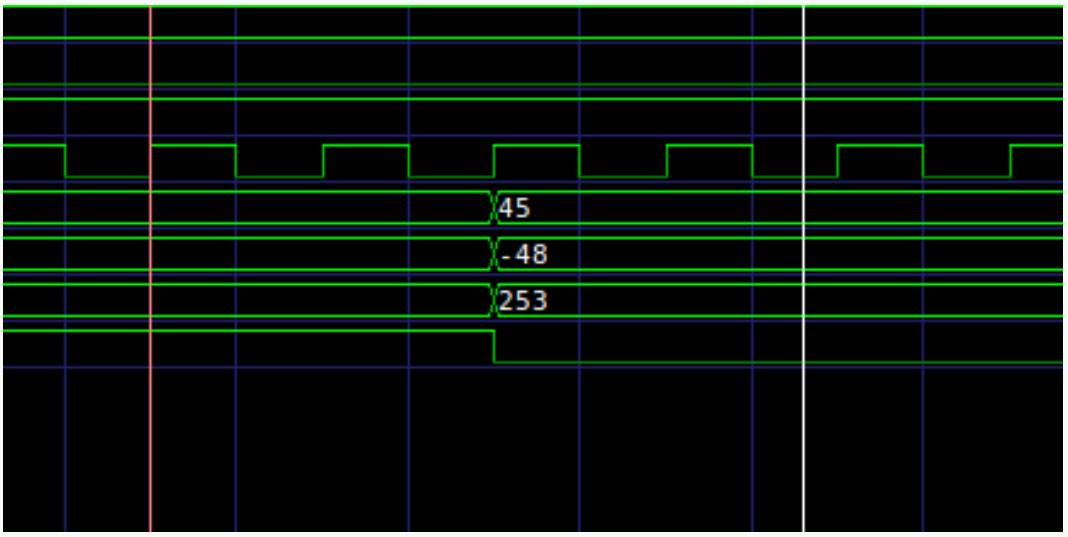


FIGURE 7 – L'additionneur fonctionne pour les nombres signés

3 Script de compilation

Solution

Script de compilation GHDL (donné sous Moodle) :

```
echo "=> cleaning..."
rm -rf waves.ghw *.o
echo "=> analyzing VHDL files..."
ghdl -a half_adder.vhd
ghdl -a full_adder.vhd
ghdl -a adder8b.vhd
ghdl -a adder8b_tb.vhd
echo "=> elaboration..."
ghdl -e adder8b_tb
echo "=> running simulation"
ghdl -r adder8b_tb --wave=waves.ghw
echo "=> starting waveform viewer"
gtkwave waves.ghw chrono.sav
```

Testbench (donné sous Moodle) :

```
--------------------------------------------------------------------
-- This file was generated automatically by vhdl_tb Ruby utility
-- date : (d/m/y) 05/11/2018 15:40
-- Author : Jean-Christophe Le Lann - 2014
--------------------------------------------------------------------
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity adder8b_tb is
end entity;

architecture bhv of adder8b_tb is

  constant HALF_PERIOD : time := 5 ns;

  signal clk : std_logic := '0';
  signal reset_n : std_logic := '0';
  signal sreset : std_logic := '0';
  signal running : boolean := true;

  procedure wait_cycles(n : natural) is
   begin
     for i in 1 to n loop
       wait until rising_edge(clk);
     end loop;
   end procedure;

  signal a : std_logic_vector(7 downto 0);
  signal b : std_logic_vector(7 downto 0);
  signal sum : std_logic_vector(7 downto 0);
  signal cout : std_logic;

begin
  --------------------------------------------------------------------
  -- clock and reset
  --------------------------------------------------------------------
  reset_n <= '0','1' after 666 ns;

  clk <= not(clk) after HALF_PERIOD when running else clk;

  --------------------------------------------------------------------
```

4 Utilisation de generate

Solution

L'instruction `generate` permet de créer des instances de composants de manière itérative, ce qui est très utile pour structurer des designs réguliers comme un additionneur N bits.

```
library ieee;
use ieee.std_logic_1164.all;

architecture using_generate of adder8b is
  signal dummy_carry : std_logic;
  signal carry : std_logic_vector(7 downto 0);
begin

  dummy_carry <= '0';

  Loop_index: for i in 0 to 7 generate

    first_stage: if i=0 generate
      stage_0: entity work.full_adder(logic)
      port map(
        a => a(i),
        b => b(i),
        cin => dummy_carry,
        sum => sum(i),
        cout => carry(i)
      );
    end generate;

    other_stages: if i>0 generate
      state_i: entity work.full_adder(logic)
      port map(
        a => a(i),
        b => b(i),
        cin => carry(i-1),
        sum => sum(i),
        cout => carry(i)
      );
    end generate;

  end generate;

  cout <= carry(7);

end using_generate;
```

TD 9 – Électronique Numérique

Conception d'un accélérateur matériel pour le calcul d'un polynôme

Notes et Conseils

On se propose ici de concevoir un accélérateur matériel, dédié au calcul de polynômes de rang 3. L'étude commence par une réflexion algorithmique, qui suggère des optimisations. Nous poursuivons la démarche par la manipulation abstraite de cet algorithme, à l'aide de *graphes*. L'accélérateur conçu par la suite se présente comme une microarchitecture, composée classiquement d'un automate de contrôle et d'un chemin de données. On se limitera à des valeurs de x entières, codées sur 32 bits.

1 Réflexion algorithmique

La conception d'un accélérateur matériel naît souvent de la création d'un algorithme aux performances théoriques intéressantes (ex : compression par Transformée en Cosinus Discret), ou par la découverte d'une reformulation intéressante (ex : multiplication de Karatsuba [1]). Nous allons ici, en miniature, suivre cette démarche.

On considère un polynôme d'ordre 3 :

$$P_3(x) = a_3x^3 + a_2x^2 + a_1x + a_0 \tag{1}$$

Questions

- **Question 1 :** Combien de multiplications et d'additions sont-elles nécessaires ?
- **Question 2 :** À l'aide du schéma de Hörner, normalement connu, montrer que l'on peut réduire le nombre de ces opérations.

Solution

Question 1 : Il en faut : $3 + 2 + 1 = 6$ multiplications, et 3 additions.

Question 2 : On peut réécrire le polynôme précédent de la manière suivante :

$$\begin{aligned} P_3(x) &= x(a_3x^2 + a_2x + a_1) + a_0 &(2)\\ &= x(x(a_3x + a_2) + a_1) + a_0 &(3)\end{aligned}$$

On en déduit que sous cette forme, seulement **3 multiplications** sont nécessaires (en plus des 3 additions).

2 Flot de données

L'étude du flot de données (ou DFG : *dataflow graph*) permet d'établir les dépendances du calcul. On représente généralement ces dépendances à l'aide d'un graphe orienté : chaque nœud

1. Nous vous invitons à vous renseigner sur ces deux curiosités mathématiques

représente une opération (ici arithmétique) ou une entrée ou une sortie. Chaque arc entre deux nœuds est étiqueté par le nom de la variable qui circule du nœud source au nœud destination. Dans notre cas, le graphe est acyclique. Il indique clairement l'enchaînement des opérations, mais il s'agit encore d'une vision abstraite du calcul, indépendante de choix d'architecture matérielle. On peut toutefois noter que ce DFG peut être vu comme une première tentative de réalisation purement combinatoire du circuit (mais son chemin critique est très certainement rédhibitoire).

Questions

Question 3 : Établir le graphe de flot de données issu du schéma de Hörner.

Solution

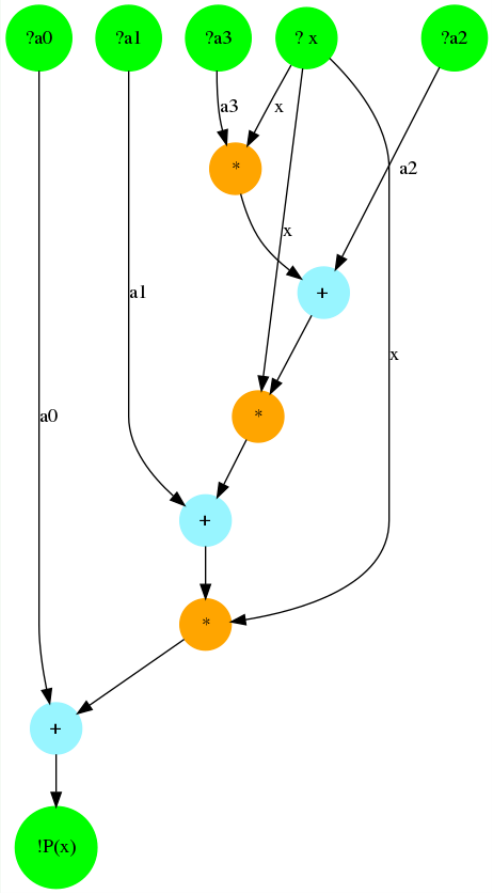


FIGURE 1 – Graphe de flot de données (DFG) du schéma de Hörner

3 Ordonnancement. Assignation et partage des ressources

La notion d'**ordonnancement** permet de découper le DFG en différentes étapes de calculs, appelées **control-step** (CSTEP). Prosaïquement, cela revient à dessiner différentes lignes qui représentent effectivement cette découpe du DFG.

Chacune des étapes s'exécute en un seul cycle d'horloge. Selon la durée physique du cycle d'horloge, il est toutefois possible de réaliser des découpes du DFG plus ou moins "longues" : plus le cycle est long, plus il est possible d'envisager de **chaîner** un ensemble d'opérations. À la fin du cycle, on devra veiller à échantillonner les données produites : formulé autrement, on peut dire que la découpe en CSTEPs détermine où devront se trouver les *registres intermédiaires* de calcul.

En général, les opérateurs de calculs et les registres de calcul représentent la majeure partie de la surface occupée sur silicium. Il est souvent souhaitable de les utiliser avec parcimonie et notamment de les partager dès que cela est possible. On réalise l'**assignation** des opérations aux opérateurs afin d'établir où se réaliseront effectivement les calculs du DFG abstrait. Bien évidemment, à la fin d'un CSTEP tous les opérateurs utilisés sont à nouveau disponibles pour le CSTEP suivant, ce qui donne des opportunités de partage de ressources entre opérations ordonnancées sur deux CSTEPs différents. Ces informations d'assignation peuvent être également annotées sur le DFG : à côté de l'opération abstraite du DFG, on peut indiquer sur quel opérateur matériel elle s'exécutera.

Avant même la phase d'assignation, le concepteur se donne par ailleurs un objectif en terme de nombre de ressources utilisées au total : c'est la phase d'**allocation de ressources**. Il peut se donner le droit d'utiliser tel nombre de multiplieurs, tel nombre d'additionneurs, tel nombre de registres, etc. Parmi les contraintes, on retrouve d'autres paramètres variés et en particulier la fréquence attendue du circuit. On se fixe ici une fréquence d'horloge de 100 MHz.

Questions

- **Question 4 :** On s'alloue 1 multiplieur et un additionneur. Le multiplieur a un temps de calcul de 7 ns et l'additionneur de 2 ns. Concernant les registres, on considère que leur temps de setup et hold sont très inférieurs, et non significatifs par rapport aux opérateurs. Proposer un ordonnancement qui vous semble judicieux. À combien de CSTEPs aboutissons-nous ?
- **Question 5 :** Réaliser l'assignation des ressources et notamment des registres.

Solution

Question 4 : On voit sur le DFG que le calcul de multiplication et d'addition se suivent. Le chemin critique de cet enchaînement est de $7+2 = 9$ ns. Or, la fréquence de fonctionnement des registres est de 100 MHz soit un temps de cycle de 10 ns : ce budget de 10 ns laisse parfaitement le temps aux signaux de traverser à la fois le multiplieur et l'additionneur, dans un même cycle d'horloge. Cela est d'autant plus vrai que les temps de setup et hold sont considérés comme négligeables dans l'énoncé. On peut donc chaîner les opérateurs matériels dans le même cycle. Par contre, à la fin de cette portion (répétée) de circuit, il faudra échantillonner les signaux, dans des bascules D (registres). Le schéma suivant donne 3 CSTEPs. On suppose que les entrées et sorties sont également "clockées" (échantillonnées), ce qui est une bonne pratique.

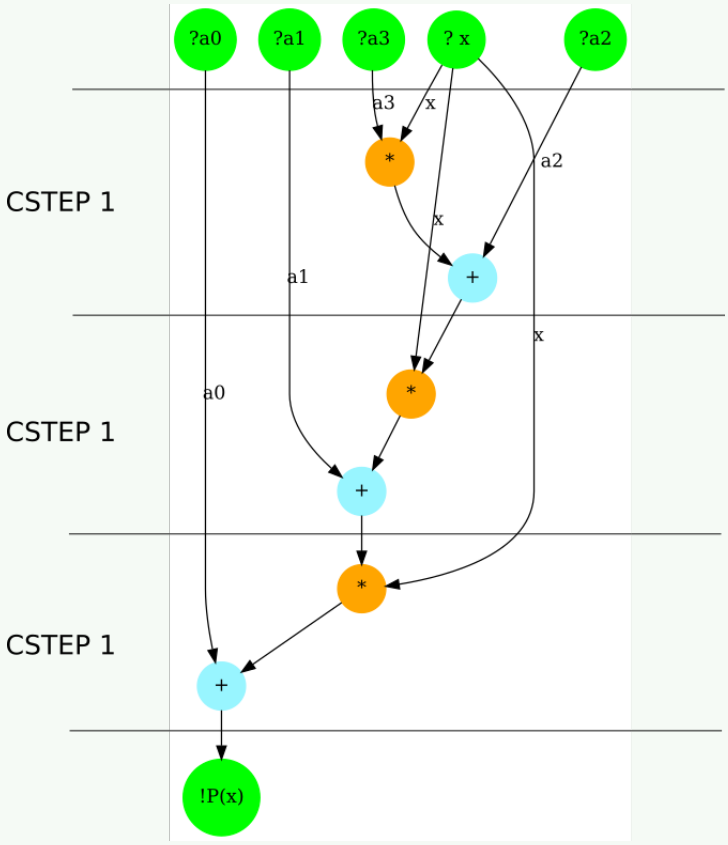


FIGURE 2 – DFG ordonnancé en 3 CSTEPs

Solution

Question 5 : On annote les ressources sur le schéma : il s'agit désormais d'opérateurs matériels et non plus d'opérations abstraites. Ces opérateurs peuvent être réutilisés à chaque cycle (CSTEP). De même un registre dont on n'a plus besoin peut être réutilisé. C'est le cas du registre R3, qui contient initialement le premier coefficient lu. Après cette lecture, on peut utiliser R3 pour les calculs des variables temporaires (intermédiaires) de calcul.

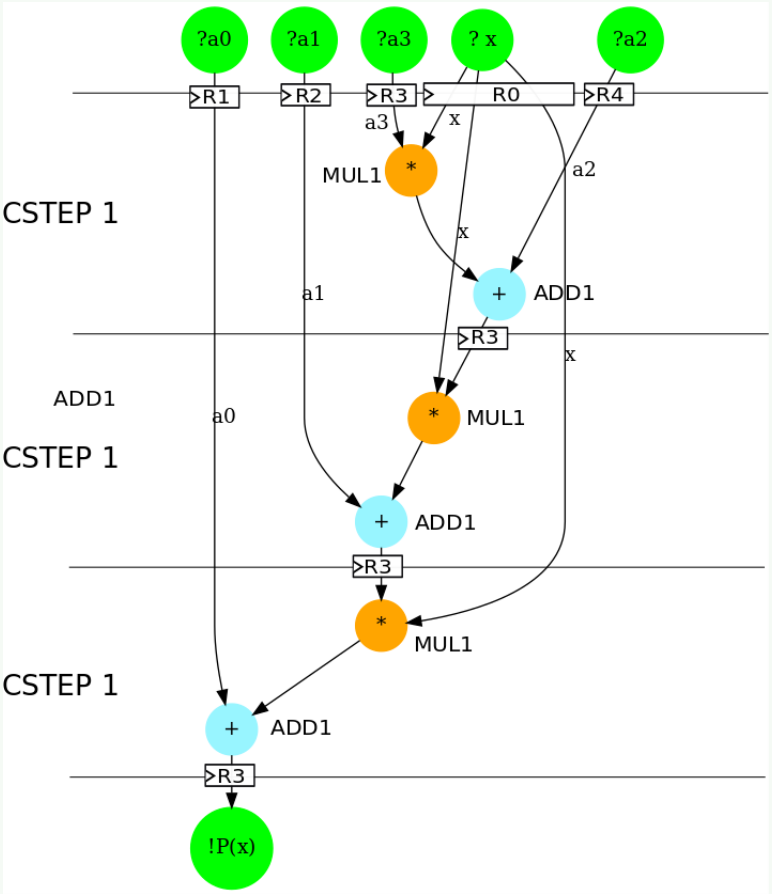


FIGURE 3 – DFG avec assignation des ressources

4 Microarchitecture : automate de contrôle et datapath

Dès lors qu'un ordonnancement est choisi, il est alors aisé de décrire un automate et un chemin de donnée. L'automate permet de séquencer les différentes étapes. Il réalise en outre, pour chacune d'entre elles, les actions appropriées. À l'issue de l'assignation, ces actions reviennent à contrôler les multiplexeurs d'un chemin de données (datapath) : ces multiplexeurs servent soit à échantillonner une donnée à l'entrée d'un registre, soit à router une donnée vers un opérateur matériel.

Questions

- **Question 6 :** Dessiner l'automate de contrôle.
- **Question 7 :** Dessiner un chemin de données.
- **Question 8 :** Compléter l'automate pour indiquer les actions réalisées dans chaque état.

Solution

Question 6 :

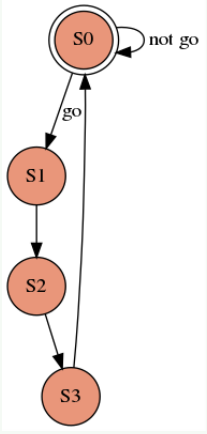


FIGURE 4 – Automate de contrôle

Solution

Question 7 : Pour déterminer le chemin de données (datapath), on observe le DFG ordonnancé et alloué (question 5). Il s'agit essentiellement de déterminer les opérandes des opérateurs. Prenons le Multiplieur MUL1 : on voit que ses opérandes sont R0 et R3 lors du CSTEP 1, et à nouveau R0 et R3 aux CSTEPs 2 et 3. Le chemin entre les registres et l'entrée de MUL1 est donc très simple et direct. Observons maintenant ADD1 :

- dans le CSTEP 1, ADD1 opère sur le résultat combinatoire de MUL1 et sur R4
- dans le CSTEP 2, ADD1 opère sur le résultat combinatoire de MUL1 et sur R2
- dans le CSTEP 3, ADD1 opère sur le résultat combinatoire de MUL1 et sur R1

On peut en déduire que pour assurer ces 3 fonctionnements, il est nécessaire d'adjoindre un multiplexeur sur le second opérande de ADD1, dont les sources sont R4, R2 et R1. Notons que nos deux opérateurs sont commutatifs, ce qui a grandement simplifié notre analyse. On en déduit une version, encore incomplète, du datapath, présentée sur la figure suivante.

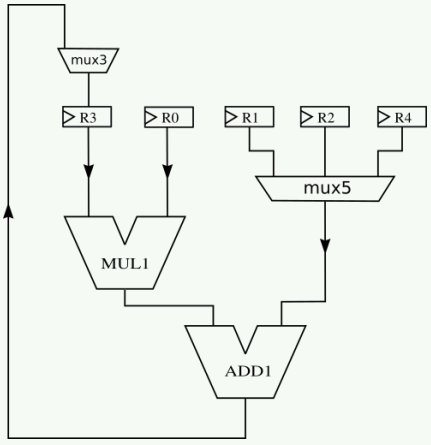


FIGURE 5 – Datapath primitif (incomplet)

Solution

Suite Quelques détails restent à régler : comment sont acheminés les entrées et calculs intermédiaires, vers ces registres ? Il faut bien penser à l'échantillonnage des entrées et leur stockage tout au long du calcul. Là encore on repose sur un multiplexeur, dont la commande peut jouer ce rôle. Afin de ne pas modifier la donnée stockée (potentiellement ré-échantillonnée à chaque cycle), on fait ainsi "recirculer" la donnée de la sortie Q à l'entrée D des bascules.
Désormais notre datapath est complet. Reste à déterminer les instants de contrôles des multiplexeurs, émis par le contrôleur (FSM).

a_2

FIGURE 6 – Datapath complet

On numérote les entrées des multiplexeurs : ces valeurs correspondront aux valeurs du signal de commande qu'il faudra envoyer au multiplexeur pour qu'il fasse passer l'entrée correspondante sur sa sortie. Cela nous permettra de piloter le cheminement des données (question suivante).

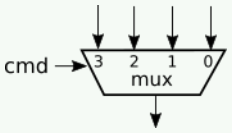


FIGURE 7 – Numérotation des entrées des multiplexeurs

Solution

Question 8 : On se propose de présenter cet automate sous la forme tabulée suivante. Ce tableau fait apparaître les *transferts de registres*, ainsi que les micro-actions correspondantes, sur les multiplexeurs.

État	**Effet algorithmique**	**Transferts de registres**	**Contrôle Mux**
S0	Arrivée des données	$R0 \leftarrow x$ $R1 \leftarrow a_0$ $R2 \leftarrow a_1$ $R3 \leftarrow a_3$ $R4 \leftarrow a_2$	$cmd_0 \leftarrow 1$ $cmd_1 \leftarrow 1$ $cmd_2 \leftarrow 1$ $cmd_3 \leftarrow 1$ $cmd_4 \leftarrow 1$ $cmd_5 \leftarrow 0$
S1	$t_1 = a_3 \times x + a_2$	$R3 \leftarrow (R3 \times R0) + R4$	$cmd_0 \leftarrow 0$ $cmd_1 \leftarrow 0$ $cmd_2 \leftarrow 0$ $cmd_3 \leftarrow 2$ $cmd_4 \leftarrow 0$ $cmd_5 \leftarrow 0$
S2	$t_2 = t_1 \times x + a_1$	$R3 \leftarrow (R3 \times R0) + R2$	$cmd_0 \leftarrow 0$ $cmd_1 \leftarrow 0$ $cmd_2 \leftarrow 0$ $cmd_3 \leftarrow 2$ $cmd_4 \leftarrow 0$ $cmd_5 \leftarrow 1$
S3	$P(x) = t_2 \times x + a_0$	$R3 \leftarrow (R3 \times R0) + R1$	$cmd_0 \leftarrow 0$ $cmd_1 \leftarrow 0$ $cmd_2 \leftarrow 0$ $cmd_3 \leftarrow 2$ $cmd_4 \leftarrow 0$ $cmd_5 \leftarrow 2$

Quelques remarques :

— On notera au passage que ces transferts de registres ressemblent à ce que réalise un langage de programmation de bas niveau : l'assembleur. Toutefois, nos transferts de registres sont ici plus puissants que l'assembleur : plusieurs transferts peuvent avoir lieu en même temps, en parallèle, alors qu'en assembleur traditionnel chaque transfert se fait séquentiellement.

— L'opération de multiplication suivie d'une addition est si fréquente dans le domaine scientifique, qu'il existe un opérateur dédié : le **MULACC**. Il est présent notamment dans les processeurs de traitement du signal, appelés DSP (*digital signal processors*, les plus connus étant ceux de Texas Instruments). Les FPGA modernes possèdent en plus de leur matrice de LUTs, un ensemble de tels opérateurs optimisés.

5 Dimensionnement des registres

Une difficulté récurrente lorsqu'on traduit un algorithme en microarchitecture réside dans le dimensionnement des signaux, c'est-à-dire la détermination du nombre de bits nécessaires à la représentation des nombres, sans perte d'information préjudiciable. On parle aussi de leur *dynamique*. Il y a perte d'information lors d'une troncature (élimination brutale de bits à un certain rang).

Questions

- **Question 9 :** Soient deux signaux s_n et $s_{n'}$ entiers codés respectivement sur n et n' bits. Déterminer le nombre de bits nécessaires à l'addition $s_n + s_{n'}$, sans perte d'information.
- **Question 10 :** Même question pour $s_n \times s_{n'}$.
- **Question 11 :** En déduire la dynamique des signaux apparaissant dans le datapath.

Solution

Question 9 : Par exemple, avec $n = 8$ et $n' = 8$, $s_n, s_{n'} \in \{0 \dots 255\}$ et $s_n + s_{n'} \in \{0 \dots 510\}$. On se convainc facilement que le nombre de bits pour le résultat de l'addition est $\mathbf{max(n, n') + 1}$.

Question 10 : Pour la multiplication le résultat est $\mathbf{n + n'}$.

Question 11 : À la fin de chaque CSTEP, la dynamique des signaux calculés augmente. Si chaque entrée est codée sur un même nombre n de bits :

- CSTEP 1 : R3 nécessite $2n + 1$ bits
- CSTEP 2 : R3 nécessite $(2n + 1) + n + 1 = 3n + 2$ bits
- CSTEP 3 : R3 nécessite $(3n + 2) + n + 1 = 4n + 3$ bits

À titre d'exemple, partant de $n = 8$ bits, on obtient **35 bits** nécessaires en sortie.

6 Codage VHDL

Le code VHDL de cet accélérateur est fourni sur Moodle, ainsi qu'un banc de test. La syntaxe de VHDL est complexe et nouvelle pour vous, mais vous êtes à même d'en comprendre les grandes lignes.

Questions

- **Question 12 :** Observez attentivement le code de l'accélérateur. Dessiner l'architecture générale du système. Discutez avec votre encadrant afin de bien vérifier que vous avez compris la correspondance avec vos schémas précédents.
- **Question 13 :** Observez attentivement le code du banc de test. Discutez avec votre encadrant afin de bien vérifier que vous avez compris son fonctionnement.
- **Question 14 :** La simulation ne sera pas réalisée en séance, mais une vidéo sera mise à disposition.

Solution

Question 12 :

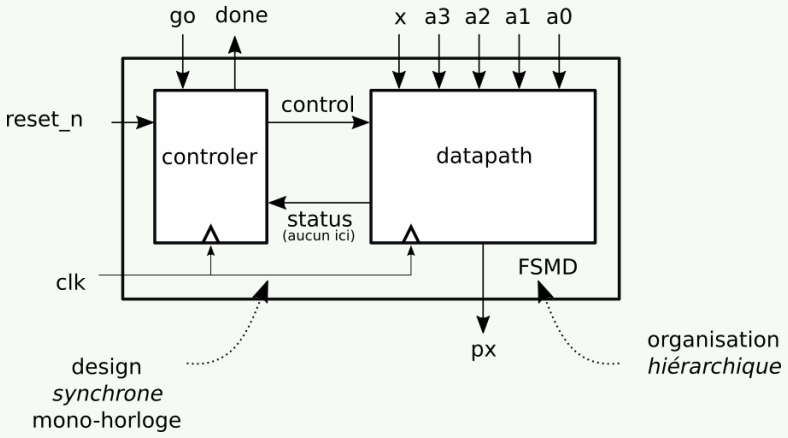


FIGURE 8 – Architecture générale du système (FSMD)

Solution

Question 13 : Le banc de test se présente comme un design classique : il est composé d'une entité et d'une architecture. Toutefois l'entité ne possède aucun port d'entrée-sortie. Cela est parfaitement normal, car notre testbench représente une "expérience virtuelle", dans un laboratoire tout aussi virtuel, entièrement équipé et autonome ! Il n'a besoin d'aucune information extérieure. L'architecture représente la paillasse où l'on prépare un "setup expérimental" : au centre, on retrouve bien entendu le circuit à tester, et tout autour de lui des instruments virtuels, sous forme de processus qui fonctionnent en parallèle, les uns des autres : générateurs de signaux, sondes, outils de diagnostique etc.
Notre testbench réalise également une comparaison des signaux issus de notre circuit (`done` et `px`) avec un modèle de référence. Ce modèle de référence est ici calculé également par VHDL. Dans notre cas, VHDL est en effet capable de calculer un polynôme, de manière "informatique". Il aurait été assez maladroit de recourir à un modèle de référence Python ou Matlab pour écrire une référence si simple. Toutefois, cela arrive fréquemment que les équipes en présence n'aient pas la même culture. Les algorithmiciens, qui fournissent de tels modèles de références, connaissent rarement VHDL.

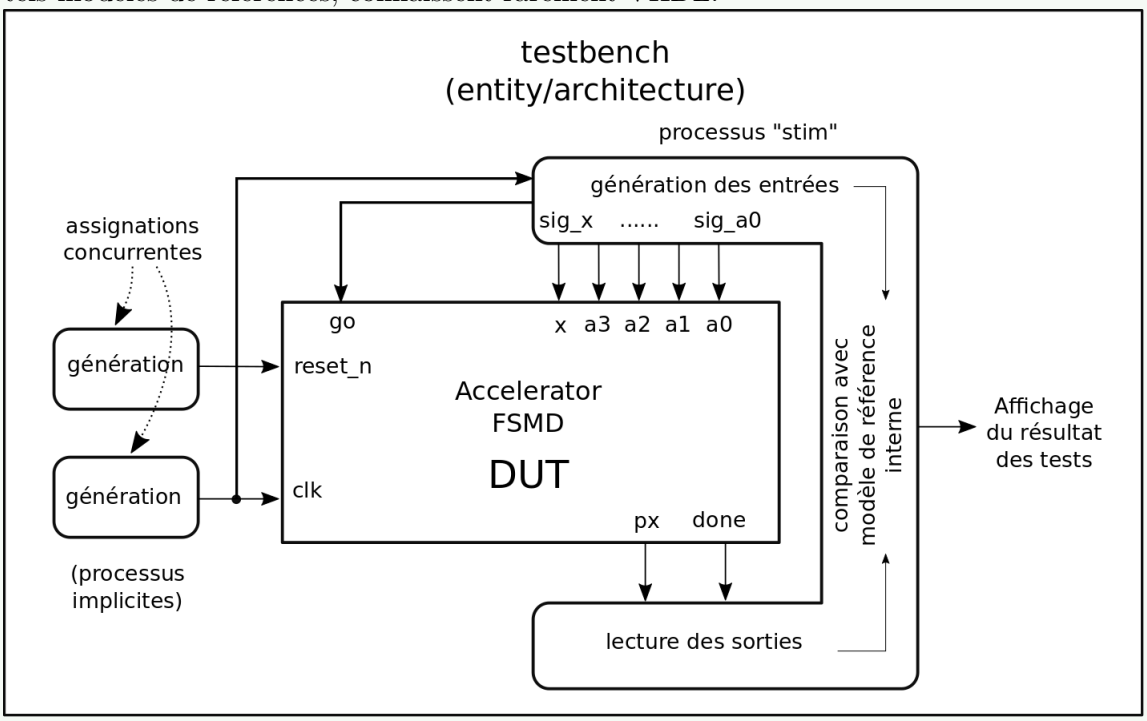


FIGURE 9 – Architecture du banc de test

Solution

Question 14 : La simulation se fait ici sous Linux, à l'aide de GHDL. GHDL permet de générer un simulateur compilé. Cette génération se fait en 3 grandes étapes :

- **Analyse** des fichiers avec l'option `-a` de ghdl.
- **Élaboration** du simulateur avec l'option `-e` de ghdl.
- **Lancement** (**run**) du simulateur avec l'option `-r` de ghdl.
- **Visualisation** des résultats de simulation avec GtkWave.

Ces commandes répétitives sont facilement automatisables sous Linux. Un script bash `compile.x` est fourni sur Moodle.

On obtient le **chronogramme** suivant :



FIGURE 10 – Chronogramme de simulation commenté

Solution

Dans le terminal Linux apparaissent les messages du simulateur, ainsi que les “reports” que l’on a décrits en VHDL :

```
~/JCLL/ens/UE1.2-Electronique/TD/9_BE_Horner/VHDL$ ./compile.x
============ cleaning  ============
======= analyzing vhdl files ======
=> controler_pkg.vhd
=> controler.vhd
=> datapath.vhd
=> fsmd.vhd
=> fsmd_tb.vhd
======= elaborating simulation ======
=> fsmd_tb
========= running simulation ========
@0ms   :(report note): running testbench for fsmd(rtl)
@0ms   :(report note): waiting for asynchronous reset
@215ns :(report note): p(x)=4x^3 +3x^2 + 2x + 1
@215ns :(report note): sending parameters a0...a3
@315ns :(report note): expected p(0)=1
@365ns :(report note): actual : p(0)=1
@365ns :(report note): expected p(1)=10
@415ns :(report note): actual : p(1)=10
@415ns :(report note): expected p(2)=49
@465ns :(report note): actual : p(2)=49
@465ns :(report note): expected p(3)=142
@515ns :(report note): actual : p(3)=142
@515ns :(report note): expected p(4)=313
@565ns :(report note): actual : p(4)=313
@565ns :(report note): expected p(5)=586
@615ns :(report note): actual : p(5)=586
@615ns :(report note): expected p(6)=985
@665ns :(report note): actual : p(6)=985
@665ns :(report note): expected p(7)=1534
@715ns :(report note): actual : p(7)=1534
@715ns :(report note): expected p(8)=2257
@765ns :(report note): actual : p(8)=2257
@765ns :(report note): expected p(9)=3178
@815ns :(report note): actual : p(9)=3178
@815ns :(report note): expected p(10)=4321
@865ns :(report note): actual : p(10)=4321
@865ns :(report note): expected p(11)=5710
@915ns :(report note): actual : p(11)=5710
@915ns :(report note): expected p(12)=7369
@965ns :(report note): actual : p(12)=7369
@965ns :(report note): expected p(13)=9322
@1015ns:(report note): actual : p(13)=9322
@1015ns:(report note): expected p(14)=11593
@1065ns:(report note): actual : p(14)=11593
@1065ns:(report note): expected p(15)=14206
@1115ns:(report note): actual : p(15)=14206
@1115ns:(report note): expected p(16)=17185
@1165ns:(report note): actual : p(16)=17185
@1165ns:(report note): expected p(17)=20554
@1215ns:(report note): actual : p(17)=20554
@1215ns:(report note): expected p(18)=24337
@1265ns:(report note): actual : p(18)=24337
@1265ns:(report note): expected p(19)=28558
@1315ns:(report note): actual : p(19)=28558
@1315ns:(report note): end of simulation
@1315ns:(report note): number of errors 0 / 20 tests
============== viewing ==============
GTKWave Analyzer v3.3.86 (w)1999-2017 BSI
[0] start time.
```